

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL
ESCOLA POLITÉCNICA
CURSO DE BACHARELADO EM ENGENHARIA DE SOFTWARE
AGES – AGÊNCIA EXPERIMENTAL DE ENGENHARIA DE SOFTWARE

JOÃO PEDRO SALLES DA SILVA

**MEMORIAL DE ATUAÇÃO NA AGÊNCIA EXPERIMENTAL DE ENGENHARIA DE
SOFTWARE - PERÍODO 2023 A 2026
AGES I, II, III e IV**

Porto Alegre

2026

Dedicatória

Dedicatória: Texto no qual o autor do trabalho oferece homenagem ou dedica o seu trabalho a alguém.

AGRADECIMENTOS

Os agradecimentos devem ser dirigidos àqueles que contribuíram de maneira relevante à elaboração do trabalho, restringindo-se ao mínimo necessário, como instituições (CNPq, CAPES, PUCRS, empresas ou organizações que fizeram parte da pesquisa), ou pessoas (profissionais, pesquisadores, orientadores, etc.).

Os agradecimentos devem ser colocados de forma hierárquica de importância e para trabalhos financiados com recursos de instituições (CAPES, CNPq, FINEP, FAPERGS, etc.) os agradecimentos são obrigatórios a essas instituições.

“ Epígrafe: É um item onde o autor apresenta a citação de um texto que seja relacionado com o tema do trabalho, seguido da indicação de autoria do mesmo. (texto iniciando do meio da página alinhado a direita) “

Autor da Epígrafe

RESUMO

Neste relato, compartilho minha jornada na AGES (Agência Experimental de Engenharia de Software), começando como AGES I, onde minha principal responsabilidade era contribuir para o desenvolvimento e testes do software. Ao progredir para a posição de AGES II, assumi um papel mais aprofundado na construção e no aprimoramento do banco de dados, bem como na implementação de novas funcionalidades. Na AGES III, ajudei a produzir toda a arquitetura de software do projeto, além de desimpedir e ajudar em questões técnicas aqueles que têm mais dificuldades e os AGES IV fazem parte da gerência.

PALAVRAS CHAVES: AGES, Engenharia de Software, Aprendizado, Projeto, Desenvolvimento.

LISTA DE ILUSTRAÇÕES

Figura 1 – Time do projeto Informativo para Imigrantes e Refugiados	11
Figura 2 – Protótipo no Figma - AGES I	13
Figura 3 – Time do projeto Let Me Trial	20
Figura 4 – Protótipo no Figma - Let Me Trial	22
Figura 5 – Time - Se Doce Fosse	30
Figura 6 – Banco de Dados	32
Figura 7 – Diagrama de Arquitetura	33
Figura 8 – Protótipo no Figma - Home	34
Figura 9 – Protótipo no Figma - Página de Produtos	35
Figura 10 – Protótipo no Figma - Tabela de pedidos (Admin)	36
Figura 11 – Protótipo no Figma - Página Admin	36
Figura 12 – Time - Uma Porto Alegre Alemã	46
Figura 13 – Diagrama do Banco de Dados	48
Figura 14 – Protótipo no Figma - Landing Page	50
Figura 15 – Protótipo no Figma - Mapa	51
Figura 16 – Protótipo no Figma - Detalhe no Mapa	52
Figura 17 – Protótipo no Figma - Detalhe de Edificações	53

LISTA DE TABELAS

LISTA DE SIGLAS

AWS	Amazon Web Services	32
------------	-------------------------------	----

SUMÁRIO

1	APRESENTAÇÃO DA TRAJETÓRIA DO ALUNO	9
2	AGES I — IMIGRANTES E REFUGIADOS	10
2.1	Introdução	10
2.2	Desenvolvimento do Projeto	11
2.3	Atividades Desempenhadas Pelo Aluno no Projeto	14
2.4	Conclusão	17
3	AGES II — “LET ME TRIAL”	19
3.1	Introdução	19
3.2	Desenvolvimento do Projeto	20
3.3	Atividades Desempenhadas Pelo Aluno no Projeto	23
3.4	Conclusão	27
4	AGES III — “SE DOCE FOSSE”	29
4.1	Introdução	29
4.2	Desenvolvimento do Projeto	30
4.3	Atividades Desempenhadas Pelo Aluno no Projeto	37
4.4	Conclusão	43
5	AGES IV — “UMA PORTO ALEGRE ALEMÃ”	45
5.1	Introdução	45
5.2	Desenvolvimento do Projeto	47
5.3	Atividades Desempenhadas Pelo Aluno no Projeto	54
5.4	Conclusão	57
6	CONSIDERAÇÕES FINAIS	58
	Referências	59

1 APRESENTAÇÃO DA TRAJETÓRIA DO ALUNO

Minha trajetória na área de Software inicia antes da faculdade. Aos meus 11 anos jogava Minecraft e era fascinado em como os plugins deixavam o jogo muito mais atrativo e diferente e comecei a buscar na internet maneiras de editar e criar um plugin próprio. Foi neste momento que me deparei com a linguagem Java, no começo foi estranho e diferente, mas consegui concluir alguns projetos e fiquei muito feliz por isso.

Não continuei na área e minhas pretensões eram outras, mas na hora de escolher meu curso, movido pela paixão e por aquele sentimento do passado, resolvi entrar no curso de Engenharia de Software no segundo semestre de 2022. Desde então procuro evoluir como frontend e sigo um entusiasta da área.

Em maio de 2023, participei da Hackatona de Engenharia de Software na PU-CRS, onde deveríamos resolver uma solução com o tema “Como utilizar a Inteligência Artificial para promover o bem social?”. Nosso grupo apresentou a solução da Comunicação Universal, projeto que consiste em um tradutor de libras em tempo real. Obtivemos o segundo lugar entre 14 equipes.

2 AGES I — IMIGRANTES E REFUGIADOS

2.1 Introdução

O projeto Informativo para Imigrantes e Refugiados teve como objetivo principal desenvolver um aplicativo mobile para facilitar o acesso à informação para pessoas imigrantes e refugiadas. Os stakeholders Éfren Alvarado, Alex Guilherme e Henrique trouxeram como problema a dificuldade que esse público enfrenta para encontrar informações confiáveis sobre direitos, oportunidades de educação e programas de acolhimento disponíveis no Brasil.

A proposta do aplicativo é reunir essas informações em um único lugar, permitindo que ONGs, universidades, escolas e outras instituições cadastrassem programas com detalhes suficientes para orientar melhor os usuários. Dessa forma, o produto buscava reduzir a barreira de acesso à informação e apoiar a integração social e educacional de imigrantes e refugiados.

O projeto foi desenvolvido durante o segundo semestre de 2023, com início em 4 de agosto de 2023 e conclusão em 8 de dezembro de 2023, sob orientação do professor Dilnei Venturini. A Figura 1 apresenta o time envolvido no projeto.

Figura 1 – Time do projeto Informativo para Imigrantes e Refugiados



Fonte: wiki do projeto

2.2 Desenvolvimento do Projeto

2.2.1 Repositório do Código Fonte do Projeto

O código fonte do projeto foi organizado em repositórios no GitLab da AGES, separados entre *backend*, *frontend* e *wiki*. Essa divisão ajudou o time a manter responsabilidades mais claras: o *backend* concentrava as regras de negócio e as APIs, o *frontend* reunia o aplicativo mobile e a *wiki* servia como espaço de documentação do projeto, decisões e materiais de apoio.

Durante as sprints, também utilizei pela primeira vez em um projeto o versionamento com Git (Git Community, 2025) para subir minhas alterações, abrir branches e permitir que os colegas mais experientes revisassem o código. No início eu ainda tinha pouca prática com esse fluxo, mas ao longo do projeto percebi a importância de fazer commits com mais frequência para receber feedback mais cedo.

Repositório para o *backend*

Repositório para o *frontend*

Repositório para a *wiki*

2.2.2 Banco de Dados Utilizado

O banco de dados utilizado no projeto foi o SQL Server. Ele foi incorporado principalmente nas sprints finais e tinha o papel de persistir as informações consumidas pela aplicação, como cadastros, programas ofertados e dados necessários para que o *backend* respondesse às requisições feitas pelo aplicativo.

O time não disponibilizou imagens e figuras sobre a modelagem do banco de dados, mas a estrutura geral seguia um modelo relacional, com tabelas para usuários, programas, instituições e outras entidades relevantes para o funcionamento do aplicativo. Para mim, essa foi a primeira experiência prática com um banco de dados em um projeto real, o que me ajudou a entender melhor como as informações são organizadas e acessadas por meio de APIs.

2.2.3 Arquitetura Utilizada

O *backend* foi desenvolvido em Java 11 (Gosling, 2018) com Spring Boot (Pivotal Software, 2023), seguindo uma organização em camadas. A camada de *Controller* ficava responsável por receber as requisições, a camada de *Service* concentrava as regras de negócio e a camada de *Repository* fazia a comunicação com o banco de dados. Essa separação ajudava a deixar o código mais organizado e facilitava o entendimento de onde cada responsabilidade deveria ficar.

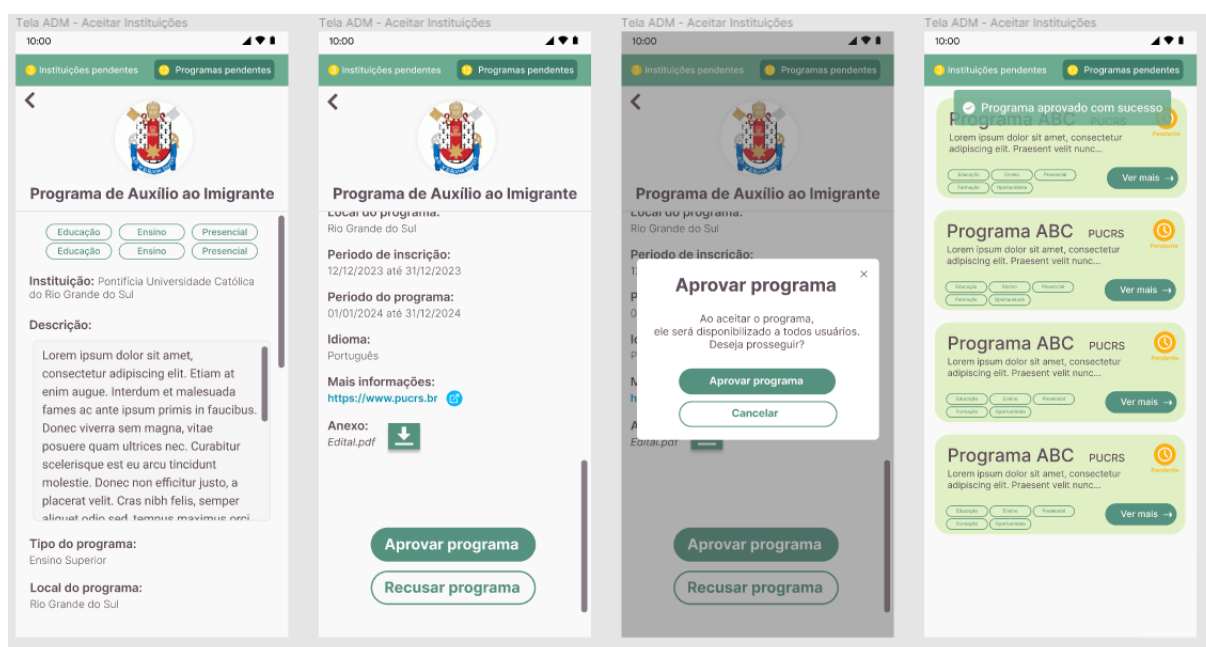
No *frontend*, o projeto foi estruturado em React Native (Facebook, 2023b) com TypeScript (Microsoft, 2023), organizado em componentes, telas, rotas, serviços, utilitários e chamadas para APIs. Essa organização permitia que o time trabalhasse de forma mais modular, facilitando a manutenção e a evolução do código ao longo das sprints. Para mim, essa foi uma oportunidade valiosa de aprender sobre arquitetura de software e como organizar um projeto de forma eficiente, já que atuei a maior parte do tempo no *frontend*.

2.2.4 Protótipos das Telas Desenvolvidas

Utilizamos o Figma (Figma, Inc., 2026) para construir os protótipos das telas e apresentar aos stakeholders a ideia inicial do aplicativo. Esses protótipos ajudaram o time a validar o fluxo de navegação, discutir a organização das informações e transformar as necessidades dos clientes em telas mais concretas antes do desenvolvimento. Para mim, como alguém que estava começando no *frontend*, o Figma também serviu como referência visual para implementar componentes e telas com mais segurança.

A Figura 2 apresenta um exemplo de protótipo utilizado durante o projeto.

Figura 2 – Protótipo no Figma - AGES I



Fonte: figma do projeto

2.2.5 Tecnologias Utilizadas

Em nosso primeiro encontro, discutimos quais tecnologias seriam utilizadas. No *backend*, o time definiu Java com Spring Boot, pois a linguagem já era conhecida por boa parte do grupo em função das disciplinas da faculdade. No *frontend*, utilizamos React Native com TypeScript, escolha adequada para um aplicativo mobile que precisava funcionar tanto em Android quanto em iOS.

Também usamos o Figma para os protótipos, o Postman para testar requisições e o GitLab para organizar código, tarefas e documentação. Para mim, essas tecnologias representaram um primeiro contato mais real com desenvolvimento de software em

equipe. Eu já tinha visto programação em sala de aula, mas ainda não tinha tido a experiência de conectar interface, API, banco de dados e principalmente trabalhar com versionamento e colaboração em um projeto real.

2.3 Atividades Desempenhadas Pelo Aluno no Projeto

2.3.1 Sprint 0

Na Sprint 0, o foco principal foi entender o projeto, alinhar o problema apresentado pelos stakeholders e definir as tecnologias que seriam utilizadas. Como era minha primeira experiência na AGES, essa sprint teve um papel muito importante de ambientação. Eu ainda estava entendendo como funcionavam as cerimônias, a divisão de tarefas e a dinâmica de trabalhar em um projeto com pessoas de diferentes níveis de experiência.

Depois da definição inicial da stack, dediquei boa parte do tempo a estudar *React Native* (Facebook, 2023b) e *TypeScript* (Microsoft, 2023), pois seria meu primeiro contato prático com essas tecnologias. Também participei da criação dos mockups no Figma, ficando responsável por telas de administrador. No começo tive dificuldade com a ferramenta, principalmente por não conhecer bem seus recursos e a forma de organizar elementos visuais, mas com a prática e com a ajuda do time consegui concluir as telas previstas.

Essa sprint me mostrou que o trabalho em grupo exigia mais do que apenas escrever código. Aprendi a dividir tarefas, acompanhar o andamento pelo processo Scrum e pedir ajuda quando necessário. Como AGES I, foi um primeiro contato importante com a rotina de desenvolvimento de software em equipe.

2.3.2 Sprint 1

Na Sprint 1, definimos as *User Stories* e nos dividimos em duas squads. Os alunos puderam escolher uma área de preferência, entre *frontend* e *backend*, e eu escolhi atuar no *frontend*. Essa escolha fazia sentido para mim justamente porque eu tinha pouca experiência nessa parte e queria aproveitar a AGES para aprender algo que ainda não dominava. Sempre tive curiosidade em desenvolvimento de aplicativos, mas nunca tinha tido a oportunidade de trabalhar com *React Native*, *TypeScript* ou

integração com APIs. Por isso, essa primeira sprint foi um momento de aprendizado intenso e de entender como rodar o *frontend* localmente, como organizar o código e como implementar telas seguindo os protótipos do Figma.

Depois da divisão das squads, os AGES III me ajudaram a fazer o setup do ambiente de desenvolvimento, instalar as dependências e rodar o aplicativo localmente. Eles também me explicaram como funcionava a estrutura do projeto, como os componentes eram organizados. Também me ajudaram a organizar as tarefas e fiquei responsável por desenvolver a tela de acolhimento (onboarding) e criar uma chamada de API para textos. A parte visual da tela foi mais tranquila, pois consegui seguir o protótipo e aplicar os conceitos iniciais de React Native. Já a integração com o *backend* foi mais difícil, porque eu ainda não entendia bem como uma API era chamada a partir do aplicativo e como testar esse fluxo com Postman.

Consegui finalizar a atividade com o apoio de colegas mais experientes, que me ajudaram a entender melhor a comunicação entre *frontend* e *backend*. Essa foi uma das primeiras vezes em que percebi, na prática, como pedir ajuda no momento certo acelera o aprendizado e evita que uma dúvida individual vire bloqueio para a entrega da sprint.

2.3.3 Sprint 2

Após uma boa entrega na Sprint 1, mantivemos as squads e eu decidi continuar no *frontend*. Dessa vez, fiquei responsável por criar um componente de anexo de arquivos no aplicativo. Comecei a sprint estudando mais sobre React Native e procurando alternativas para permitir que o usuário selecionasse documentos dentro do fluxo da aplicação.

Inicialmente tentei utilizar a biblioteca *Document Picker*, mas encontrei problemas de compatibilidade com o Expo. Essa dificuldade me travou por alguns momentos, porque eu ainda não tinha experiência suficiente para diferenciar um erro de implementação de uma limitação da biblioteca escolhida. Depois de pesquisar e conversar com colegas AGES III, entendi que o caminho mais adequado era usar o *expo-document-picker*, que se integrava melhor ao projeto e as dependências já utilizadas. Com essa troca, consegui finalizar o componente de anexo de arquivos. Por conta da dificuldade inicial, fiquei apenas com uma entrega parcial, mas consegui implementar a funciona-

lidade principal e deixá-la pronta para integração com o restante do aplicativo.

Com essa troca, consegui finalizar o componente de anexo de arquivos. A principal lição da sprint foi entender a importância de comunicar problemas cedo. Quando compartilhei a dificuldade, consegui receber direcionamento e evitar perder ainda mais tempo em uma solução que não se encaixava bem no ambiente do projeto. Mesmo

2.3.4 Sprint 3

Dando continuidade ao projeto, na Sprint 3 permaneci no *frontend* e fiquei encarregado de criar e estilizar as tabs de navegação, desenvolver a tela de meus programas e implementar um *dropdown* para os tipos de programa. Depois de quase dois meses de projeto, eu já estava mais confiante e compreendia melhor as tecnologias que estávamos utilizando, por isso consegui executar as tarefas com mais autonomia. Isso me permitiu fazer diferente da sprint passada e pegar mais de uma atividade para trabalhar, o que me ajudou a entender melhor como organizar meu tempo e priorizar tarefas.

Nas abas de navegação, utilizei o *Material Top Tabs Navigator*, do React Navigation, e trabalhei para deixá-las próximas ao que havia sido desenhado no Figma. Também criei a tela de meus programas com dados simulados, deixando a estrutura pronta para uma integração futura. Além disso, resolvi um débito técnico na tela de cadastro de programas, adicionando um *dropdown* para selecionar se o programa era de ensino básico, ensino superior ou vinculado a uma ONG. O que mais me deixou orgulhoso foi as tabs de navegação no Administrador, já que ficaram muito próximas do protótipo e consegui implementar a navegação entre as telas de forma funcional, recebendo elogio dos colegas AGES III.

Essa foi uma sprint em que me senti mais preparado tecnicamente. As tarefas foram concluídas sem grandes bloqueios e percebi uma evolução clara em relação às primeiras semanas. Também aprendi a importância de iniciar minhas tarefas mais cedo e subir commits com mais frequência no GitLab, porque isso dava tempo para os AGES III revisarem meu código e apontarem ajustes antes da entrega.

2.3.5 Sprint 4

Na Sprint 4, já na reta final do projeto, fiquei responsável por desenvolver a tela de aceitar e recusar programas, que serviria como base para a tela de aceitar e recusar instituições. Também auxiliei pontualmente na tela de ajuda e contato. Depois de três sprints trabalhando no *frontend*, eu já estava mais confortável com estilização, mas essa tarefa ainda trouxe novos desafios.

O principal problema técnico apareceu ao usar um *ScrollView* dentro de outro *ScrollView*. No início, o comportamento do scroll interno não funcionava como eu esperava, e precisei pesquisar para entender como resolver. Depois de alguns testes, consegui ajustar o comportamento com uma configuração simples, mas que eu ainda não conhecia. Também implementei um modal de confirmação para que o usuário verificasse se realmente queria aceitar um programa antes de concluir a ação.

Essa sprint foi uma das que mais me fez perceber minha evolução. A tarefa foi mais exigente do que as anteriores, mas eu já tinha repertório para pesquisar, testar e corrigir o problema sem depender tanto dos outros. Na retrospectiva final, ficou claro para mim que o período como AGES I tinha sido muito proveitoso, principalmente por eu ter saído de um conhecimento quase nulo em *frontend* para conseguir entregar telas e componentes reais do projeto.

2.4 Conclusão

O projeto Informativo para Imigrantes e Refugiados foi muito importante para meu início na AGES e no mundo real de trabalho. Os stakeholders demonstraram satisfação com o resultado, e para mim foi gratificante perceber que algumas partes do produto tinham sido desenvolvidas por mim. Como eu estava começando na área, ver uma tela ou componente meu fazendo parte de uma entrega real foi uma experiência motivadora.

Tecnicamente, minha evolução foi grande. Entrei no projeto com conhecimento quase nulo em *frontend*, sem entender bem integração entre *frontend* e *backend*, uso do Postman ou fluxo de versionamento em um projeto colaborativo. Ao longo das sprints, aprendi a desenvolver telas em React Native, testar chamadas de API, resolver problemas de bibliotecas, subir commits com mais frequência e receber feedback dos colegas.

Também evoluí bastante na parte comportamental. Eu não conseguia participar das *dailies* síncronas de terça-feira no Discord, porque estava cursando Engenharia de Requisitos no mesmo horário, mas procurei manter meus progressos e impedimentos atualizados pelo canal de texto. Essa necessidade de comunicar bem, mesmo sem estar sempre presente de forma síncrona, me fez entender melhor a importância da transparência no trabalho em equipe. Foi muito agregador entender como funciona a parte mais “burocrática” de desenvolver um software, as *dailies*, retros e *plannings* me proporcionaram experiências que não teria vivido se não fosse a AGES. Por tanto, agora saio muito mais preparado para embarcar no mercado de trabalho e começar a aplicar meus conhecimentos adquiridos nesta etapa.

O projeto por si só foi muito bom, não tenho experiências passadas para comentar com mais detalhes, mas a organização foi excelente. Acredito que o trabalho do AGES IV foi muito bom, mas aqui destaco a eficiência de todos os AGES III que contribuíram muito e foram essenciais no andamento do projeto e no crescimento dos AGES I e II. Por contrapartida, acredito que o projeto pecou um pouco na hora de designar as tarefas, pois não ficava muito explícito o que deveria ser feito, resultando em dúvidas e mais trabalho para os AGES III. Também creio que o projeto seria melhor se fosse desenvolvido web, entretanto esta é uma questão definida pelo stakeholder

3 AGES II — LET ME TRIAL

3.1 Introdução

O projeto Let Me Trial tem como objetivo facilitar o encontro entre pacientes e ensaios clínicos por meio de uma plataforma web utilizada por médicos. A ideia era permitir uma pré-triagem mais rápida, verificando se as características e doenças de um paciente eram compatíveis com os critérios de inclusão e exclusão definidos nos estudos clínicos disponíveis.

Os stakeholders Dr. Giovani Gadonski e Matheus Stortti apresentaram a necessidade de tornar esse processo mais eficiente. Sem uma ferramenta de apoio, muitos pacientes poderiam se deslocar até o hospital sem ter perfil para determinado estudo. Com uma triagem mais automatizada, seria possível descartar ou pré-encaminhar casos ainda na consulta, economizando tempo e aumentando as chances de encontrar pacientes compatíveis com os ensaios.

O projeto foi desenvolvido durante o primeiro semestre de 2024, com início em 1º de março de 2024 e conclusão em 5 de julho de 2024, sob orientação do professor Jorge Audy. A Figura 3 apresenta o time envolvido no projeto.

Figura 3 – Time do projeto Let Me Trial



Fonte: wiki do projeto

3.2 Desenvolvimento do Projeto

3.2.1 Repositório do Código Fonte do Projeto

Os repositório foram divididos entre web-medico, web-site, web-administrador, api-medico, api-administrador, auth, infra e wiki.

Repositório para o *web-site*

Repositório para o *web-medico*

Repositório para o *web-administrador*

Repositório para o *api-medico*

Repositório para *auth*

Repositório para *infra*

Repositório para a *wiki*

3.2.2 Banco de Dados Utilizado

No banco de dados, o projeto utilizou PostgreSQL (PostgreSQL Global Development Group, 2024), escolhido por ser um banco relacional robusto e adequado para representar entidades com relacionamentos claros. Diferente da minha experiência na AGES I, em que eu tive pouco contato com a persistência, na AGES II precisei entender melhor como os dados se relacionavam, porque o filtro de estudos e o cadastro de pacientes dependiam diretamente dessas informações.

As principais entidades eram *Paciente*, *Medico*, *Area*, *Estudo*, *Criterio*, *CriterioEstudo* e *Resposta*. A entidade *Paciente* armazenava os dados necessários para a triagem; *Medico* representava o profissional que utilizaria a plataforma; *Area* organizava as subáreas dos estudos; *Estudo* concentrava as informações dos ensaios clínicos; *Criterio* guardava as perguntas ou condições avaliadas; *CriterioEstudo* fazia a relação entre um estudo e seus critérios, indicando resposta esperada e se o critério era opcional; e *Resposta* registrava a resposta de um paciente para determinado critério.

// Falta adicionar figura

3.2.3 Arquitetura Utilizada

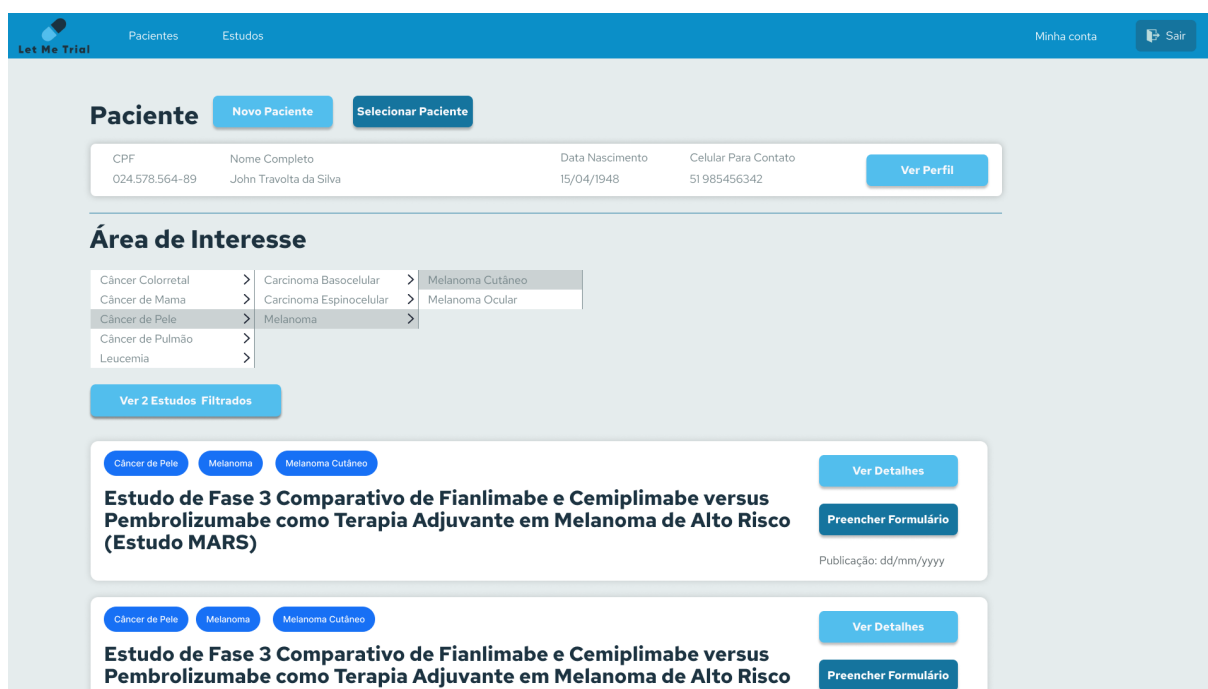
No backend, estamos utilizando a Arquitetura Hexagonal, que é um modelo de design de software que divide o sistema em três partes principais: Domínio: o coração do sistema, onde estão as regras de negócio e entidades importantes. Aplicação: coordena as ações do sistema e lida com interações entre o domínio e o mundo externo. Adaptadores: conecta o sistema ao mundo externo, permitindo interações sem que o domínio precise saber dos detalhes técnicos. No frontend, utilizamos o núcleo/core e seus submódulos, como core/domain para definir as entidades principais da aplicação, como usuários e médicos. O core/useCases é essencial para dividir a lógica de negócios em casos de uso 13 comuns e específicos para diferentes tipos de usuários, enquanto o adapters/presenter nos ajuda a adaptar os dados da lógica de negócios para a interface do usuário. Os frameworks/ui são vitais para organizar os componentes e páginas da interface do usuário, enquanto os frameworks/services facilitam a comunicação com APIs externas. Por fim, os utils são úteis para funções utilitárias, como autenticação e verificação do tipo de usuário

3.2.4 Protótipos das Telas Desenvolvidas

Os protótipos foram desenvolvidos no Figma (Figma, Inc., 2026) e serviram como referência para validar o fluxo com os stakeholders antes de implementar as telas. Eles foram especialmente importantes no início do projeto, quando ainda estávamos discutindo como seria a navegação, como o médico faria a triagem e como os estudos seriam apresentados.

Para mim, os protótipos ajudaram a transformar discussões abstratas em tarefas mais concretas de *frontend*. A Figura 4 apresenta um exemplo de protótipo utilizado no projeto.

Figura 4 – Protótipo no Figma - Let Me Trial



Fonte: figma do projeto

3.2.5 Tecnologias Utilizadas

No *backend*, utilizamos Java 21 (Gosling, 2018) com Spring Boot (Pivotal Software, 2023), além de JUnit para testes. Essa escolha foi feita porque boa parte do grupo já tinha familiaridade com Java, o que ajudaria na produtividade e na qualidade das entregas. O PostgreSQL foi utilizado como banco de dados relacional para armazenar as entidades principais do sistema, trazendo robustez.

No *frontend*, utilizamos React (Facebook, 2023a) com TypeScript (Microsoft, 2023) e Next.js (Vercel, 2026). Também usamos Material UI (Material-UI Team, 2023) para

acelerar a construção de componentes visuais e manter uma interface mais consistente. Durante as integrações, utilizei o Postman (Postman, 2023) para testar APIs antes de conectar as funcionalidades nas telas. Na sprint final, também tive contato com Docker (Docker, Inc., 2023) e com o fluxo de autenticação, o que ampliou bastante minha visão sobre como uma aplicação web funciona além da parte visual.

3.3 Atividades Desempenhadas Pelo Aluno no Projeto

3.3.1 Sprint 0

Na Sprint 0, depois da visita dos stakeholders, começamos a definir as metas do projeto e a forma de trabalho do time. Ficou acordado que teríamos *dailies* assíncronas no Discord, nas quais cada integrante deveria reportar o que tinha feito, o que faria em seguida e se existia algum impedimento. Também definimos a stack principal do projeto, com Java 21 e Spring Boot no *backend*, além de React, TypeScript e Next.js no *frontend*.

Minha atuação nessa sprint foi voltada principalmente ao fluxo e aos protótipos de tela. Fiquei mais encarregado de desenvolver o modal de *Minha Conta*, auxiliei na criação da tela de login e participei das discussões sobre outras telas. Como o produto dependia muito da experiência do médico durante a triagem, essas conversas foram importantes para alinhar o que seria simples de usar e, ao mesmo tempo, tecnicamente possível de desenvolver. Dessa vez estudei pouco tecnicamente, pois a experiência da AGES I me trouxe bagagem e conhecimento técnico, logo era mais importante focar nos protótipos e em um início de modelagem do banco de dados, para que o time pudesse começar a desenvolver a aplicação já com uma estrutura mais clara.

O cliente gostou bastante do fluxo e da aparência das telas propostas, mas ainda existia uma dúvida importante sobre como deveria funcionar o filtro de estudos. Depois de bastante discussão, chegamos a um consenso inicial. O principal problema da sprint foi a falta de organização em algumas tarefas e a demora para elas ficarem claras no *board*. Além disso, a retrospectiva acabou consumindo muito tempo, o que prejudicou a *planning* presencial, que não existiu presencialmente. Uma pena, porque acredito que a *planning* presencial teria sido muito importante para o alinhamento do time, principalmente no início do projeto.

3.3.2 Sprint 1

Começamos a Sprint 1 com a *planning* prejudicada e com os repositórios do *frontend* ainda não disponíveis. Isso trouxe um pouco de desorganização para o início do desenvolvimento e acabou sobrecarregando os AGES IV, que precisaram assumir decisões e desbloqueios além do esperado para a sua função. Mesmo assim, conseguimos nos dividir em squads: a minha tinha dois integrantes focados no *backend* e dois no *frontend*. Fiquei no *frontend* com a Carolina e demos início ao desenvolvimento.

Minha primeira tarefa foi desenvolver um componente de botão reutilizável. Apesar de ser minha primeira experiência com React na versão web, a tarefa foi relativamente tranquila e consegui entregar um componente que poderia ser usado pelos colegas em outras partes da interface. Depois disso, trabalhei no card de estudos, que foi mais complexo por envolver composição de componentes, estilização e organização das informações apresentadas ao usuário. Aproveitei também para criar um componente de *Tag*, que seria utilizado dentro do card e em outros lugares do software.

Outro ponto importante foi o desenvolvimento do filtro de estudos. No início, optei por uma solução baseada em *checkboxes* do Material UI, mas precisei estudar melhor como esses componentes funcionavam e como o estado deveria ser controlado. A maior dificuldade da sprint do grupo foi a integração com o *backend*. Como boa parte do grupo tinha pouca experiência com *frontend* web, passamos bastante tempo discutindo como realizar chamadas para a API e como estruturar os dados retornados.

Para resolver isso, estudei o uso do Axios e criei um componente *wrapper* que conectava o filtro aos cards de estudos. Esse componente fazia a chamada para a API, recebia a lista de estudos e repassava os dados para a tela. Com isso, conseguimos avançar na integração sem deixar a página inteira responsável por toda a lógica. Essa sprint reforçou meu aprendizado em React, JavaScript, integração com APIs e trabalho em equipe.

3.3.3 Sprint 2

Após apresentarmos a entrega anterior para o cliente, ficou decidido que o filtro de estudos deveria deixar de ser baseado em *checkboxes* e passar a funcionar como um *dropdown*. Também houve uma reformulação na parte de integração, pois o *frontend*

adotou uma nova abordagem para realizar chamadas à API. No começo, achei que seria uma alteração simples, já que eu havia desenvolvido o filtro anterior, mas logo percebi que a mudança exigiria mais cuidado.

Decidimos criar um *dropdown* com dois níveis, acompanhado por um botão para limpar as seleções e outro para filtrar e exibir os estudos. Usei Material UI (Material-UI Team, 2023) e estudei o *React Select* para implementar o comportamento esperado. O início foi mais lento porque eu precisava entender a biblioteca e adaptar a lógica do filtro anterior, mas depois que compreendi o funcionamento consegui avançar melhor.

A sprint também foi muito impactada pelas enchentes no Rio Grande do Sul. As aulas foram canceladas por duas semanas e depois retornaram em formato online, o que interrompeu o ritmo do projeto e deixou o time mais distante. Mesmo com esse contexto, conseguimos entregar o filtro em formato de *dropdown*, ainda com alguns ajustes pendentes, como melhorias no botão de limpar e correções de integração.

Como aprendizado, percebi que ler o código com mais paciência era tão importante quanto começar a implementar rápido. A mudança no filtro exigiu entender melhor o que já existia, reaproveitar partes úteis e aceitar que, em um cenário de crise externa, algumas entregas precisariam ser deixadas de lado para focar no essencial. A comunicação com o time também foi um desafio, pois a distância e a falta de contato presencial dificultaram o alinhamento, mas tentamos manter as *dailies* ativas e usar o Discord para esclarecer dúvidas e compartilhar avanços.

3.3.4 Sprint 3

Durante a Sprint 3, minhas principais tarefas foram ajustar o funcionamento do botão de limpar do filtro de estudos, corrigir a estilização da página de estudos e, caso sobrasse tempo, iniciar a tela de login. O botão de limpar não se comportava corretamente: depois de resetar o filtro, uma seleção anterior podia voltar a aparecer quando o usuário escolhia uma nova opção. Como o filtro era uma parte central da aplicação, esse ajuste era importante para evitar uma experiência confusa para o médico.

Consegui corrigir o reset do filtro e deixar a funcionalidade pronta para entrar na *branch* principal. Também ajustei a estilização da página de estudos, principalmente problemas de espaçamento e largura relacionados aos cards. Essa parte foi mais tranquila porque eu já tinha mais familiaridade com HTML (World Wide Web Consortium

(W3C), 2023b) e CSS (World Wide Web Consortium (W3C), 2023a), além de estar mais confortável com a estrutura do projeto.

Como ainda sobrou tempo, comecei a desenvolver a tela de login. Combinei com o time que, nessa sprint, eu faria apenas a parte estática da tela, deixando a integração com a *api-auth* para a sprint seguinte. A partir do modelo no Figma, implementei o layout e aprendi a criar a função de mostrar ou ocultar senha usando o ícone de olho aberto e fechado, além de entender melhor o funcionamento de formulários no React.

Essa sprint aconteceu ainda em um contexto afetado pelas enchentes, com parte do trabalho sendo conduzida remotamente. Mesmo assim, percebi como uma comunicação clara ajudava o time a continuar produtivo em casa. Para mim, foi uma das melhores sprints do projeto, porque consegui concluir minhas tarefas, adiantar uma funcionalidade importante e ainda ficar disponível para ajudar colegas quando necessário. No fim, ficou pendente a parte de integração do login, que seria discutida com os AGES IV e AGES III.

3.3.5 Sprint 4

Na Sprint 4, depois de uma apresentação ao cliente, fomos informados de que não poderíamos seguir utilizando dados sensíveis do paciente, como nome completo, CPF e telefone. Por questões de privacidade e LGPD //falta add sigla, reorganizamos o cadastro para trabalhar com apelido, CEP e sexo. A partir disso, fiquei responsável pelo *remap* dos dados do paciente no *frontend*, o que exigiu alterações em diversos arquivos.

Comecei substituindo nome por apelido, uma mudança mais simples por envolver campos de texto parecidos. A alteração de CPF para CEP foi mais trabalhosa, porque havia validadores e formatadores que precisavam ser ajustados. Ao adicionar o campo de sexo, criei um *dropdown* no cadastro de pacientes e revisei a estilização do componente para manter a interface consistente com os cards e com o restante da página.

A parte mais difícil da sprint foi integrar a tela de login com o sistema de autenticação. Foram várias horas de estudo em meio ao final do semestre, mas consegui fazer a autenticação funcionar utilizando a *Context API* do React. Depois disso, criei uma rota privada, permitindo que o usuário acessasse as páginas da aplicação apenas

quando o *token* fosse válido e estivesse ativo. Caso contrário, ele era redirecionado para a tela de login. Como ainda não havia um fluxo de cadastro dentro da aplicação, parte da criação de usuários precisou ser feita manualmente.

Essa sprint foi conturbada porque ainda sentíamos os impactos das enchentes, e o calendário acadêmico ficou mais apertado com provas e trabalhos concentrados no final do semestre. Mesmo assim, considero que o trabalho foi muito positivo. Aprendi mais sobre autenticação, Keycloak, rotas privadas, manipulação de *token* e tive meu primeiro contato prático com Docker (Docker, Inc., 2023). Foi uma sprint exigente, mas também uma das mais importantes para minha evolução técnica.

3.4 Conclusão

O projeto Let Me Trial foi um marco diferente na minha trajetória de desenvolvimento de software. Em comparação com a AGES I, precisei atuar com mais autonomia e assumir responsabilidades maiores no *frontend*. Como o time tinha pouca experiência nessa área, em vários momentos precisei pesquisar, testar soluções e avançar mesmo sem ter alguém com domínio completo da tecnologia ao meu lado. Isso foi desafiador, mas contribuiu muito para meu crescimento. Pois em certos momentos atuei como líder técnico do *frontend*, tomando decisões sobre bibliotecas, organização de código e estrutura de telas. Além disso, ajudei colegas com menos experiência, compartilhando o que eu tinha aprendido e buscando soluções em conjunto.

Acredito que consegui contribuir de forma relevante dentro dos meus limites e dos limites do grupo. Desenvolvi componentes reutilizáveis, trabalhei no filtro de estudos, ajustei telas, participei de integrações e finalizei a autenticação da aplicação. Também procurei manter uma boa comunicação com o time, participar das discussões, reportar minha situação nas *dailies* e ajudar colegas sempre que possível.

Esse foi, até então, o projeto em que mais aprendi sobre *frontend*, integração com APIs e processos de software. O grupo começou com muitas dúvidas, mas evoluiu bastante ao longo das sprints. Ver os colegas aprendendo, dividindo conhecimento e conseguindo entregar funcionalidades foi uma parte muito positiva da experiência. Ao mesmo tempo, percebi que a metodologia ágil só funciona bem quando as cerimônias têm foco e quando as tarefas ficam claras para todos.

Como autocrítica do projeto, acredito que o início poderia ter sido mais organizado.

A perda de uma *planning* importante e algumas retrospectivas muito longas prejudicaram o ritmo inicial do desenvolvimento. Também senti falta de uma participação mais constante dos AGES III em alguns momentos, o que acabou gerando sobrecarga nos AGES IV e dificultando decisões mais técnicas, como deploy e organização final do ambiente. Mesmo assim, saio da AGES II mais maduro tecnicamente, com melhor entendimento de banco de dados, autenticação, integração e trabalho em equipe.

4 AGES III — “SE DOCE FOSSE”

4.1 Introdução

O projeto Se Doce Fosse visa criar um site institucional para divulgação da marca e produtos, além de um sistema de controle de estoque que permita gerenciar de forma eficiente os ingredientes utilizados na produção de doces — especialmente cookies e brownies. A empresa, fruto do sonho de uma empreendedora recém-formada em Engenharia de Alimentos, busca ampliar sua presença digital e otimizar seu processo de produção, evitando perdas por compras excessivas ou falta de insumos para atender à demanda. Os clientes Carolina Padilha e Alessandro Machado Padilha destacaram que a ferramenta será essencial para apoiar o crescimento da marca, aproximar o público-alvo por meio da internet e das redes sociais, além de melhorar o fluxo de trabalho interno. O projeto está sob a orientação do professor Dilnei Ventirini e terá início em 04/08/2025, com previsão de conclusão em 28/11/2025.

Figura 5 – Time - Se Doce Fosse

Fonte: wiki do projeto

4.2 Desenvolvimento do Projeto

4.2.1 Repositório do Código Fonte do Projeto

Os repositórios foram definidos em três principais:

Repositório para o frontend: <https://tools.ages.pucrs.br/se-doce-fosse/frontend>, onde colocamos toda a parte web do projeto. Esse repositório contém a aplicação React com a organização de componentes, estilos e scripts de build, um README com instruções de execução local e exemplos de dados para testes. Utilizamos branches para separar 'main' (release) de 'develop' (integração) e GitHub Actions para pipeline de build e testes.

Repositório para o backend: <https://tools.ages.pucrs.br/se-doce-fosse/backend>, onde colocamos a lógica de negócio com as APIs e banco de dados. Aqui ficam o projeto Spring Boot, as configurações que usam variáveis de ambiente (com um arquivo de

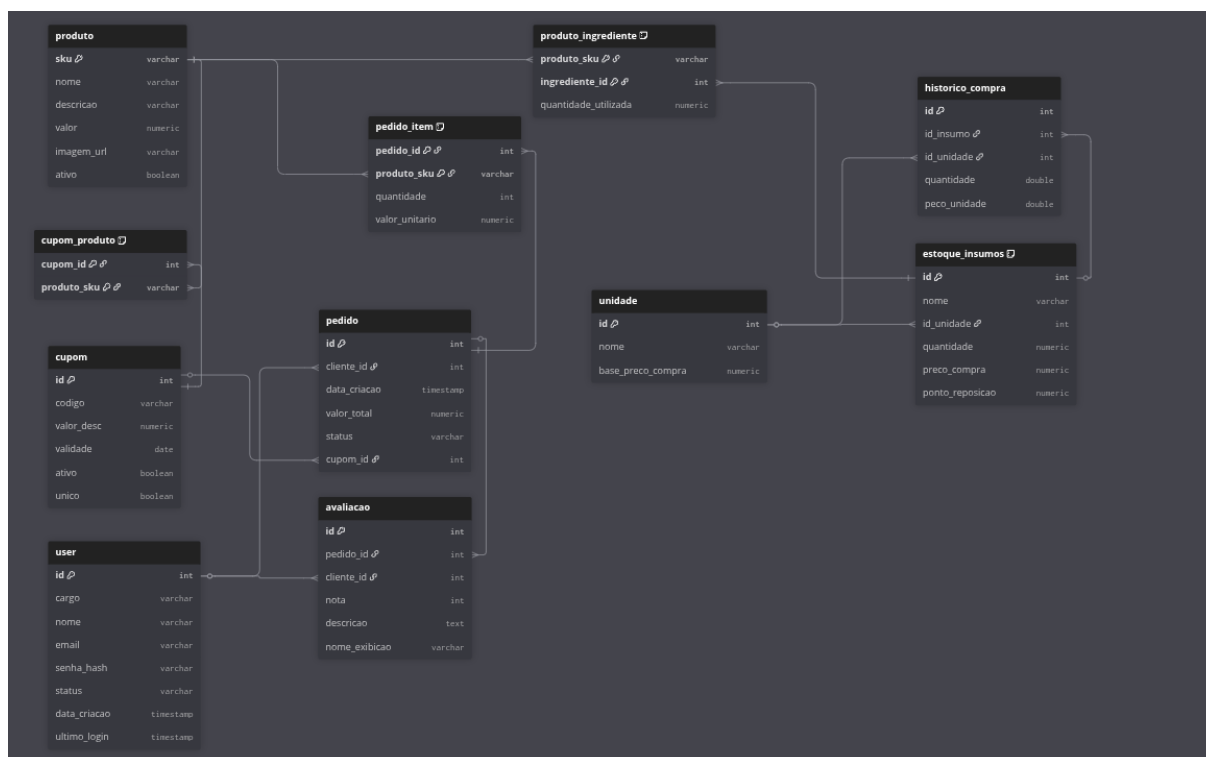
exemplo), as migrações/instruções dos bancos e os workflows de CI/CD. O fluxo de trabalho é baseado em pull requests para revisão e issues para acompanhamento das tarefas.

Repositório para a wiki: <https://tools.ages.pucrs.br/se-doce-fosse/wiki>, onde documentamos tudo que envolva o projeto de uma forma mais didática. A wiki reúne rotas da API, modelos de dados, instruções de deploy, convenções de codificação e guias rápidos de uso, servindo como referência para integrantes com diferentes níveis de experiência e quem for dar manutenção no futuro.

4.2.2 Banco de Dados Utilizado

No desenvolvimento do sistema, optou-se por uma abordagem dupla de bancos de dados, utilizando o PostgreSQL e o MongoDB, cada um desempenhando papéis complementares. O PostgreSQL (PostgreSQL Global Development Group, 2024), banco de dados relacional amplamente reconhecido por sua robustez e confiabilidade, foi empregado para armazenar informações que exigem alta integridade e consistência, como dados de usuários, controle de estoque e histórico de pedidos. Já o MongoDB (MongoDB, Inc., 2024), banco de dados não relacional orientado a documentos, foi escolhido para gerenciar elementos mais dinâmicos, como o carrinho de compras e listas temporárias, permitindo maior flexibilidade na estruturação e manipulação dessas informações. Essa combinação possibilita um equilíbrio entre desempenho, escalabilidade e integridade, explorando as vantagens específicas de cada tecnologia.

Figura 6 – Banco de Dados



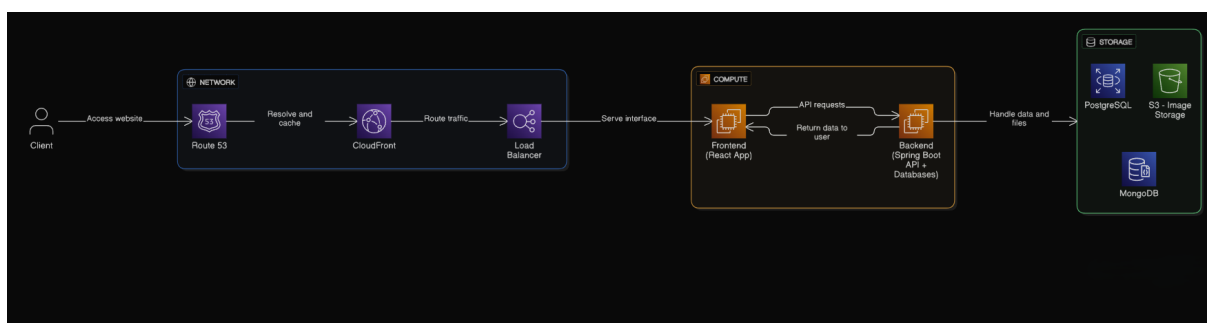
Fonte: Wiki do Projeto

4.2.3 Arquitetura Utilizada

No projeto, adotou-se uma arquitetura em camadas (*Layered Architecture*) para o desenvolvimento do backend, implementado em Java (Gosling, 2018) com o framework Spring Boot (Pivotal Software, 2023). Essa organização separa claramente as responsabilidades entre as camadas de apresentação, serviços, repositórios e entidades, promovendo maior manutenibilidade, testabilidade e clareza no fluxo de dados. No frontend, foi utilizada a biblioteca React (Facebook, 2023a), estruturada de forma modular, permitindo o reaproveitamento de componentes e a escalabilidade da aplicação. Para o estilo, optou-se pelo uso de SCSS Modules, que garantem isolamento de escopo e facilitam a manutenção visual da interface, contribuindo para uma organização consistente entre lógica, apresentação e estilos em toda a aplicação. Na parte de Infraestrutura, utilizamos a Amazon Web Services (AWS) (Amazon.com, Inc., 2025) para hospedar a aplicação, criando uma EC2 para o frontend, uma para o backend junto dos bancos de dados. O frontend utiliza NGINX para servir arquivos estáticos, atuar como reverse proxy para o backend, realizar terminação SSL/TLS, fazer cache e compressão, e gerenciar balanceamento de carga quando necessário, garantindo

desempenho e segurança. Para hospedar imagens, utilizamos o Amazon S3, que oferece armazenamento escalável e seguro para os arquivos estáticos do projeto, como imagens de produtos. Também utilizamos o GitHub Actions para realizar o CI/CD do projeto, automatizando os processos de build e testes.

Figura 7 – Diagrama de Arquitetura

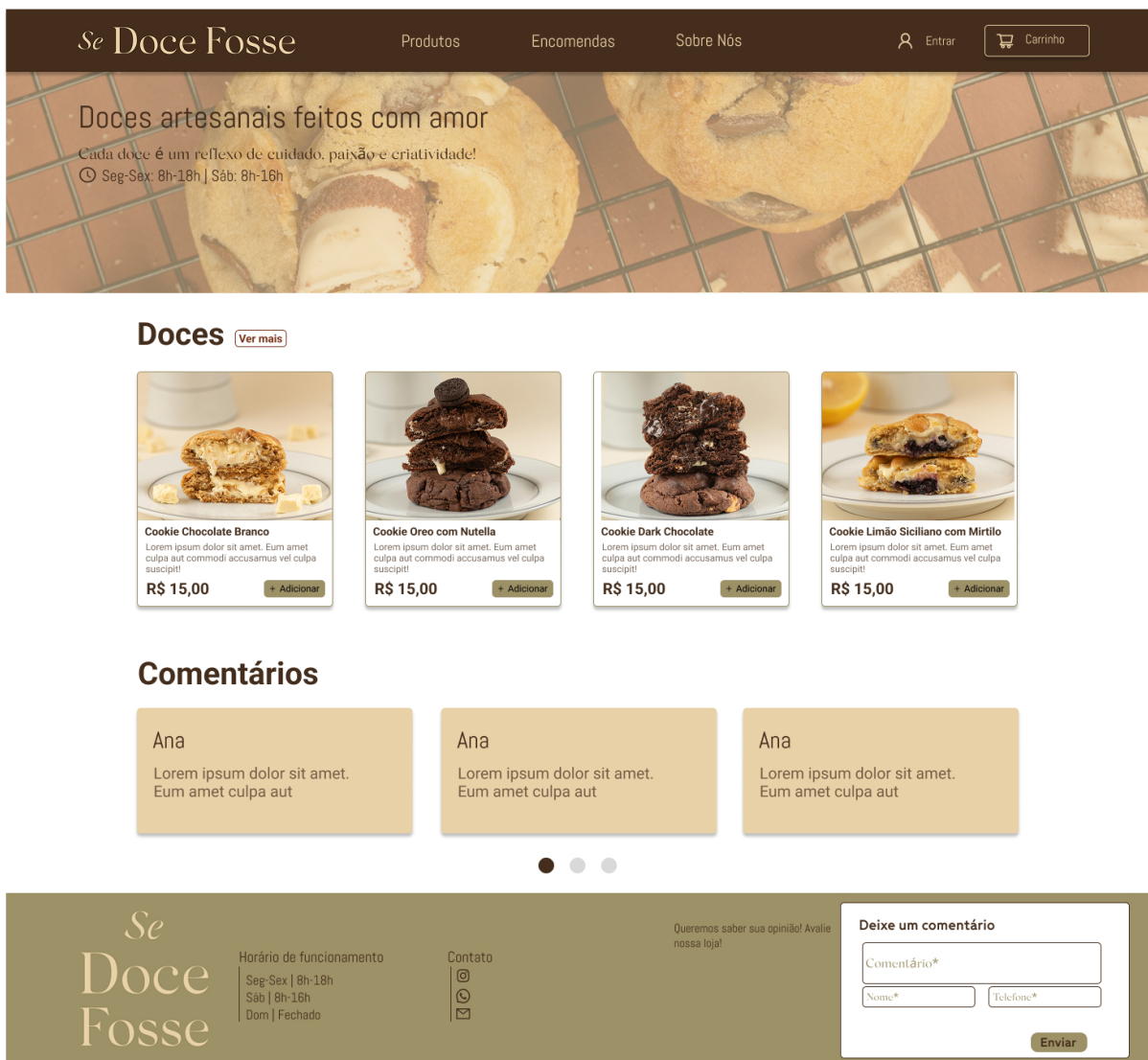


Fonte: Wiki do Projeto

4.2.4 Protótipos das Telas Desenvolvidas

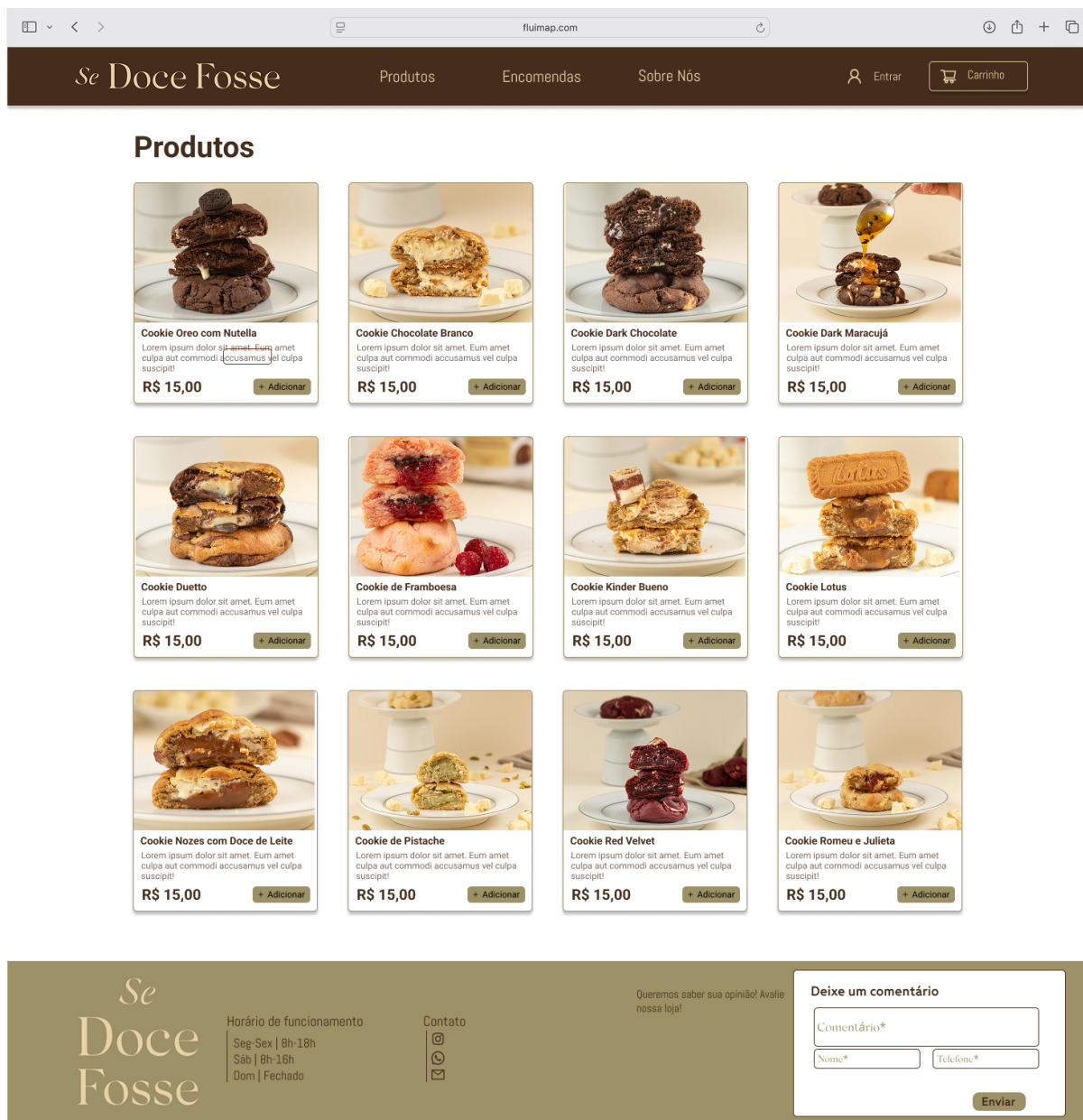
Na Sprint 0, protótipos de telas foram desenvolvidos e entregues aos clientes. Usamos elas como base para construir o frontend e validar com o clientes, além de auxiliar no backend, para entender quais endpoints seriam necessários. O protótipo foi desenvolvido no Figma, uma ferramenta colaborativa de design de interfaces que permite a criação de protótipos interativos e compartilháveis. A seguir, imagens do protótipo desenvolvido:

Figura 8 – Protótipo no Figma - Home



Fonte: Figma do projeto

Figura 9 – Protótipo no Figma - Página de Produtos



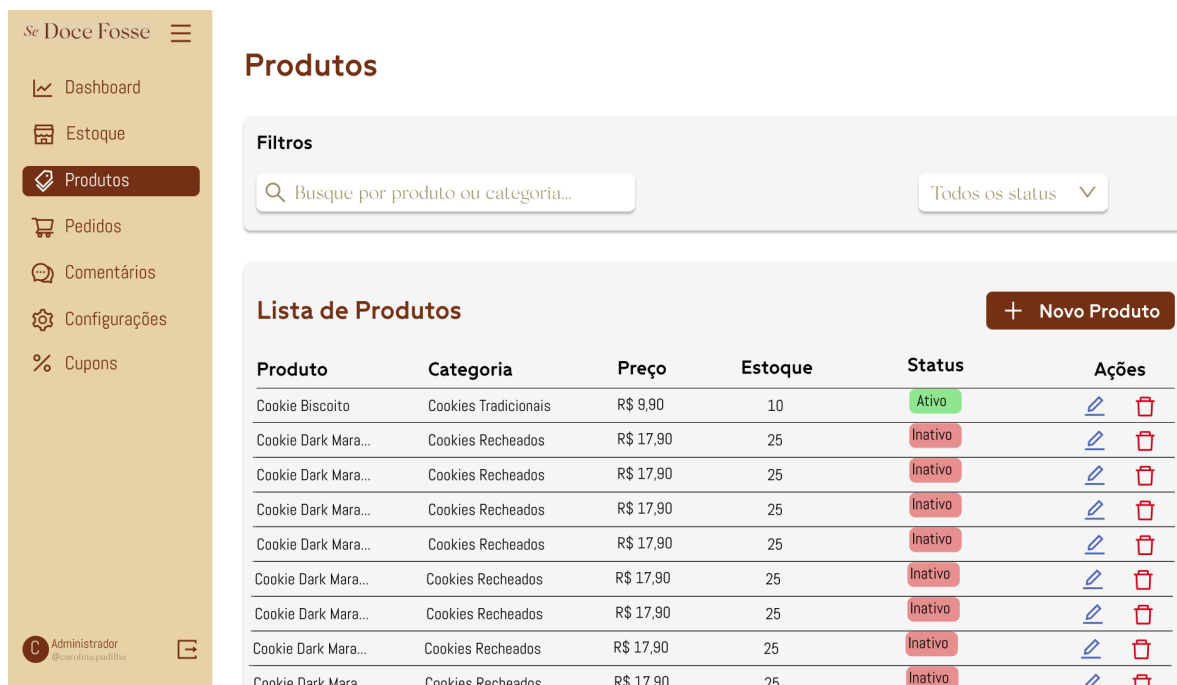
Fonte: Figma do Projeto

Figura 10 – Protótipo no Figma - Tabela de pedidos (Admin)



Fonte: Figma do Projeto

Figura 11 – Protótipo no Figma - Página Admin



Fonte: Figma do projeto

4.2.5 Tecnologias Utilizadas

Escolhemos uma stack que equilibra aprendizado e robustez: Java 21 (Gosling, 2018) com Spring Boot (Pivotal Software, 2023) no backend, PostgreSQL (PostgreSQL

Global Development Group, 2024) para dados relacionais críticos e MongoDB (MongoDB, Inc., 2024) para dados mais flexíveis, e React com TypeScript (Facebook, 2023a; Microsoft, 2023) no frontend.

Java e Spring Boot foram adotados pela familiaridade dos alunos e pela produtividade na criação de APIs testáveis. PostgreSQL garante integridade e consultas complexas, enquanto MongoDB facilita dados com esquema variável (ex.: carrinho). React + TypeScript oferece componentes reutilizáveis e verificação de tipos em desenvolvimento. Complementamos com SCSS Modules para estilos, NGINX para servir o frontend e GitHub Actions para CI/CD. A infraestrutura utiliza AWS (Amazon.com, Inc., 2025) (S3 e EC2) quando necessário. Essa combinação também expõe os alunos a paradigmas diferentes (relacional e documental) sem aumentar muito a complexidade operacional.

Essas escolhas priorizam aprendizado prático e entrega eficiente, evitando complexidade desnecessária ao mesmo tempo em que expõem os alunos a tecnologias e padrões usados em produção. Além disso, priorizamos práticas como testes automatizados e integração contínua para reforçar formação profissional e preparar os alunos para contextos reais de desenvolvimento.

4.3 Atividades Desempenhadas Pelo Aluno no Projeto

4.3.1 Sprint 0

Durante a Sprint 0, meu trabalho esteve totalmente voltado para preparar o ambiente necessário ao desenvolvimento do projeto, garantindo que toda a equipe tivesse as condições ideais para iniciar as próximas etapas. A primeira ação foi a inicialização do repositório principal, criando uma base organizada para armazenar o código-fonte do frontend e demais materiais relacionados ao sistema. Além disso, adicionei todos os membros da equipe aos repositórios, concedendo as permissões adequadas para que cada integrante pudesse colaborar de forma integrada desde o início, além de estabelecer regras para proteger as branches.

Também realizei a dockerização do ambiente do frontend, criando uma estrutura que permitisse aos integrantes responsáveis pelo backend testar a interface de forma simples e rápida, sem precisar configurar dependências locais. Configurei os arquivos e parâmetros do Docker com esse objetivo, garantindo que o front pudesse ser exe-

cutado em qualquer máquina de maneira padronizada, apenas subindo o container. Essa etapa foi essencial para reduzir barreiras entre as áreas, facilitar a integração e evitar problemas relacionados a diferenças de versões ou de configuração.

Paralelamente às tarefas mais técnicas, estudei as tecnologias ligadas à infraestrutura que seriam utilizadas ao longo do projeto. Também dediquei tempo para estruturar e documentar os processos que a equipe deveria seguir. Criei uma seção específica na Wiki chamada “Processos”, onde registrei todas as definições acordadas, incluindo fluxos de trabalho, regras para criação de branches, padrões de commits e boas práticas para organização do código. O objetivo foi garantir que ninguém tivesse dúvidas sobre como proceder, permitindo que todos soubessem exatamente quais passos adotar no dia a dia.

Ao longo dessa sprint, mantive um olhar atento para assegurar que cada detalhe estivesse finalizado antes do encerramento do ciclo. Verifiquei o funcionamento do espelhamento entre as plataformas, testei os containers Docker para confirmar que estavam executando corretamente e revisei as permissões dos membros nos repositórios. Além disso, organizei os arquivos iniciais do projeto, deixando tudo estruturado para que a equipe pudesse começar a desenvolver as funcionalidades planejadas sem enfrentar bloqueios relacionados à configuração do ambiente.

Concluindo, a Sprint 0 foi um período dedicado a preparar a base técnica e organizacional do projeto. As atividades que realizei criaram uma base sólida para o trabalho das próximas sprints. Essa preparação garantiu que o time tivesse clareza sobre as práticas adotadas e que todos dispusessem de um ambiente uniforme, seguro e pronto para receber as implementações do sistema.

4.3.2 Sprint 1

Na Sprint 1, foquei em organizar o trabalho do frontend e em dar suporte aos AGES I e II, garantindo que todos pudessem avançar sem bloqueios. Logo no início, separei as tarefas de acordo com o nível de cada integrante, definindo o que seria mais adequado para os AGES I e para os AGES II. Essa divisão ajudou bastante a dar clareza sobre as responsabilidades e permitiu que o time tivesse uma visão mais concreta do que precisava ser entregue.

Além da organização das tasks, trabalhei diretamente na implementação de alguns

componentes essenciais para o projeto, como o *Input* e o *Textarea*. Esses elementos serviram como base para o restante da interface e já deixaram o repositório com exemplos práticos para os outros integrantes, e também já deixava pronto esses componentes que seriam reutilizados em tarefas futuras. Também revisei, comentei e aprovei diversos Pull Requests, ajudando a manter a qualidade e a consistência do código que estava sendo incorporado ao projeto.

Outro ponto marcante dessa sprint foi o tempo dedicado a desimpedir colegas que estavam com dificuldades para entender o funcionamento do frontend. Muitos dos integrantes, especialmente os do AGES I, estavam tendo contato com essas tecnologias pela primeira vez e se sentiram um pouco perdidos. Para ajudar, organizei “aulas” no Discord, ferramenta que utilizamos como base para comunicação, onde expliquei o passo a passo de como os componentes funcionam, apresentei a estrutura do projeto e mostrei boas práticas para a escrita de código. Em vários momentos também fiz *pair programming*, acompanhando o desenvolvimento em tempo real e orientando sobre a melhor forma de resolver problemas específicos, além de ajudar a entender funções específicas do TypeScript.

Percebi que esse apoio constante fez diferença no engajamento do grupo. À medida que as dúvidas eram resolvidas, os colegas conseguiam evoluir mais rápido e participar ativamente das entregas. Ainda assim, notei que apenas explicações ao vivo não eram suficientes para todos, por isso, comecei a pensar na ideia de criar materiais em vídeo que complementassem a documentação já disponível, tornando o aprendizado mais acessível e facilitando o trabalho de quem prefere revisar os conteúdos de outra maneira.

Enquanto essas atividades aconteciam, também iniciei estudos sobre a infraestrutura que pretendemos usar na AWS. Esse planejamento começou de forma paralela, para que as decisões sobre deploy e hospedagem não atrasem o projeto nas próximas sprints.

De forma geral, a Sprint 1 foi muito voltada a facilitar o caminho dos colegas e a consolidar a base do frontend. Consegui equilibrar a produção de código com o suporte técnico, garantindo que todos tivessem condições de avançar. O trabalho de orientação, revisão e organização foi essencial para criar um ambiente colaborativo e produtivo, preparando o time para as próximas etapas do desenvolvimento.

4.3.3 Sprint 2

Na Sprint 2, foquei em desimpedir os AGES I e os AGES II e em começar a implementar soluções que facilitassem as próximas etapas do projeto. Logo no começo identifiquei os principais pontos de bloqueio no frontend e passei a orientar os colegas de forma assíncrona e direta, respondendo dúvidas no Discord e revisando trechos de código quando necessário. Paralelamente, comecei a levantar opções de deploy e participei das discussões iniciais sobre a arquitetura na AWS para que as decisões de infraestrutura não atrasassem as entregas.

Uma das frentes centrais foi a implementação da autenticação no frontend. Trabalhei na proteção de rotas e criei mecanismos de segurança para a área de ADMIN. Além disso, padronizei as chamadas para a API, estabelecendo convenções que tornaram as integrações mais previsíveis e fáceis de entender para os integrantes menos experientes. Essas padronizações foram documentadas de forma simples para servir como referência e reduziram dúvidas recorrentes durante a sprint.

Durante a sprint ajudei efetivamente os AGES I e AGES II em suas tasks no frontend. Desta vez dediquei mais tempo à assistência assíncrona, mas ainda assim consegui transferir conhecimento e orientar boas práticas. Também contribuí na definição de diretrizes iniciais para o deploy na AWS, alinhando expectativas com o time e apontando os caminhos a seguir.

Enfrentei problemas ao proteger algumas rotas: ao aplicar a proteção, as rotas chegavam a aparecer vazias e o conteúdo não era renderizado corretamente. Para contornar isso, resolvi mover a lógica de proteção para o layout do admin, evitando sobrecarregar as rotas e permitindo que o conteúdo fosse carregado e autorizado no nível do layout. Essa solução se mostrou simples e eficaz para o caso que estávamos enfrentando.

Como lição, ficou claro que nem tudo precisa sair perfeito desde o início. Aprendi a resolver problemas em condições não ideais e que uma comunicação rápida e objetiva muitas vezes resolve bloqueios que pareceriam complexos. De forma geral, a Sprint 2 foi importante para consolidar autenticação, padronizar chamadas de API e desbloquear o time para que as entregas pudessem evoluir com mais estabilidade.

4.3.4 Sprint 3

Na Sprint 3, novamente concentrei meus esforços em desimpedir os AGES I e II, atuando tanto na parte de API quanto no código do frontend. Não foi necessário gravar o vídeo previsto para as integrações, pois os colegas compreenderam rapidamente os conceitos após as explicações, exemplos práticos e sessões de acompanhamento que realizei. Assumi tarefas de integração entre frontend e backend: padronizei as chamadas, ajustei formatos de dados quando necessário e validei fluxos básicos com ferramentas de teste (ex.: requisições manuais e mocks), o que permitiu que várias funcionalidades passassem a se comunicar corretamente com a API. Embora o deploy final não tenha sido concluído nesta sprint, avançamos bastante no planejamento da entrega e em testes locais colaborativos com os demais dos AGES III.

Durante o processo enfrentei problemas de integração relevantes: o backend ainda apresentava instabilidades e, em alguns momentos, alterações em campos da tabela de produtos não eram persistidas, impedindo a finalização de funcionalidades no frontend. Para identificar a causa, conversei com a equipe de backend, inspecionei logs, testei endpoints isoladamente e adotei verificações intermediárias no frontend que evidenciaram quais valores não chegavam corretamente ao banco. Também houve dependência de decisões sobre reorganização dos bancos de dados, o que exigiu ajustes na arquitetura a serem feitos antes do deploy definitivo.

Além disso, realizei pair programming com AGES I e revisei pull requests para manter a qualidade do código. Essas ações reduziram o impacto dos problemas e mantiveram o ritmo das entregas, além de gerar material e exemplos que facilitaram o aprendizado dos colegas.

As lições aprendidas nesta sprint reforçaram que frontend e backend devem caminhar mais alinhados desde o início, com contratos de API claros e testes de integração contínuos. A prática de escrever e executar testes, mesmo simples e manuais nas fases iniciais, ajudaria a detectar inconsistências e evitar retrabalhos que serão feitos. Em suma, a Sprint 3 foi produtiva para desbloquear o time, consolidar integrações e avançar no planejamento do deploy, as pendências remanescentes servem agora como pontos de ação claros para as próximas sprints.

4.3.5 Sprint 4

Na Sprint 4 atuei diretamente auxiliando os AGES I nas integrações do frontend. Todos finalizaram as integrações com sucesso e passaram a rodar por conta própria.

Também trabalhei nas integrações da tabela da tela de estoque e concluí a funcionalidade de produto, incluindo a gestão de ingredientes por meio do modal de criar e editar produtos. Com isso foi possível aplicar descontos diretamente na tabela de estoque de forma consistente. Corrigi problemas de CSS relacionados a modais e revisei pull requests, aceitando mudanças corretas e solicitando ajustes quando necessário. Durante a sprint executei testes manuais de ponta a ponta e preparei exemplos de dados que facilitaram o trabalho dos colegas menos experientes.

Posteriormente finalizei o deploy na AWS, configurei o ambiente para rodar com NGINX e apontei corretamente para o backend. Isso permitiu testar a aplicação em um ambiente mais próximo ao de produção e identificar pontos finos que não eram visíveis em testes locais. Na integração inicial houve um bloqueio de CORS. As requisições foram impedidas até que o endereço do frontend, o IP elástico, fosse incluído nas liberações de CORS no backend. Identificamos a causa, ajustamos as regras de CORS e atualizamos variáveis de ambiente. Após o redeploy do backend os testes em produção passaram a funcionar normalmente e, em seguida, realizamos a demonstração ao cliente. A apresentação foi bem sucedida e o deploy foi validado pelo cliente.

Como lição, ficou evidente que decisões de infraestrutura na nuvem, como provisionamento de IP e regras de CORS, devem ser priorizadas mais cedo no cronograma, pois resolver esses pontos tardiamente gera retrabalho e atrasos para a entrega final do projeto. Sendo assim, um planejamento antecipado dessas etapas em sprints passadas, ajudaria a mitigar riscos e garantir uma entrega mais fluida sem contratempos técnicos de última hora.

Ao mesmo tempo a prioridade em apoiar o AGES I foi justificável porque desde que entrei no projeto, meu maior objetivo era passar conhecimento para quem não tinha tanta experiência, pois sei que isso faz uma diferença enorme no resultado final da AGES, já que a disciplina é feita para os alunos aprenderem e crescerem tecnicamente. Foi justamente na AGES que tive meu primeiro contato com desenvolvimento web e sem esse apoio inicial, talvez eu não teria conseguido chegar onde cheguei

hoje.

4.4 Conclusão

O projeto Se Doce Fosse representou um marco significativo na minha trajetória como desenvolvedor de software, proporcionando um ambiente desafiador e enriquecedor para aplicar e expandir meus conhecimentos técnicos, já que dessa vez atuei como Arquiteto de Software e Líder técnico do Frontend. Entender melhor sobre infraestrutura e realizar um deploy na AWS foram experiências valiosas que complementaram minha formação e me prepararam para desafios futuros na área de desenvolvimento de software. Durante o projeto, enfrentei diversos desafios técnicos e de comunicação que exigiram soluções criativas e colaborativas. A necessidade de alinhar as expectativas entre as equipes de frontend e backend, bem como a gestão de tarefas entre os diferentes níveis de experiência dos membros do time, foram aspectos cruciais que contribuíram para meu crescimento profissional.

Sobre o projeto em si, acredito que consegui contribuir de maneira significativa para o sucesso das entregas, especialmente ao apoiar os AGES I e AGES II no frontend e na resolução de bloqueios técnicos, minha maior meta era fazer com que o aprendizado deles não fosse deixado de lado, pois como já visto na primeira reunião, eles nunca tinham tido contato com esse tipo de tecnologia antes e creio que a AGES não é só sobre entregar um projeto, mas sim sobre o aprendizado e crescimento de todos os envolvidos. Um dos maiores problemas era ter um número alto de AGES IV, já que isso acabava por nenhum deles ter o controle total do projeto, fazendo com que um dependesse do outro para resolver problemas, o que não é o ideal, já que precisamos tomar decisões mais assertivas o quanto antes. Esse problema seria solucionado se eles mesmo tivessem sido designados cada um para um tipo de tarefa, como por exemplo, um para infraestrutura, outro para backend e outro para frontend, assim cada um teria total controle sobre sua área e poderia tomar decisões mais rápidas e eficazes.

Em resumo, o projeto Se Doce Fosse foi uma experiência transformadora que ampliou meus horizontes técnicos e interpessoais. As lições aprendidas sobre liderança técnica fizeram com que eu me tornasse um profissional mais completo e preparado para enfrentar os desafios do desenvolvimento de software em equipe. Ademais, a

oportunidade de ensinar alguém com pair programming foi extremamente gratificante, pois pude ver o crescimento dos colegas e a evolução do projeto como um todo, já que no final, os AGES I estavam realizando tarefas de maneira autônoma, o que demonstra o sucesso do processo de aprendizado e colaboração implementado ao longo do projeto, que era minha meta inicial.

5 AGES IV — “UMA PORTO ALEGRE ALEMÃ”

5.1 Introdução

O projeto Uma Porto Alegre Alemã tem como objetivo desenvolver um *website* com mapa digital multilíngua (Português, Alemão e opcionalmente Inglês) para divulgar, no Brasil e na Alemanha, as obras do arquiteto alemão Theodor Wiederspahn localizadas em Porto Alegre. A proposta foi pensada para aproximar o público em geral desse patrimônio histórico por meio de uma navegação acessível, organizada e com conteúdo de valor cultural.

Além do mapa principal, o sistema prevê links para cada edificação com informações mais aprofundadas, como histórico, análise estilística, desenhos técnicos, fotos e modelos tridimensionais. Também está previsto um gerenciamento de conteúdo, permitindo a criação e edição de categorias e materiais multimídia, de forma que a plataforma possa ser atualizada continuamente conforme a evolução do projeto cultural.

Os *stakeholders* do projeto são Maria Alice Dias, Márcio D’Avila e Raquel Lima, da Escola Politécnica (Curso de Arquitetura e Urbanismo), em uma iniciativa conectada ao contexto de valorização do patrimônio e de aproximação institucional. O projeto está sob orientação do professor José Pedro Schardosin e está sendo desenvolvido no semestre 2026/1, nas sextas-feiras, das 19:15 às 22:30 (6LM6NP), com previsão de entrega para o dia 3 de julho de 2026.

Figura 12 – Time - Uma Porto Alegre Alemã



Fonte: wiki do projeto

5.2 Desenvolvimento do Projeto

5.2.1 Repositório do Código Fonte do Projeto

No projeto, organizamos os materiais em repositórios separados para *frontend*, *backend* e *wiki*, o que facilitou bastante o acompanhamento das entregas e a divisão das responsabilidades entre os times. Na *wiki*, centralizamos os links e a documentação principal para manter o contexto técnico acessível para todos, incluindo decisões de arquitetura, organização das tarefas e alinhamentos com os clientes.

Também estruturamos o trabalho com foco em rastreabilidade, conectando *user stories*, tarefas técnicas e entregas por sprint. Essa organização foi importante para eu conseguir acompanhar o que estava pendente no painel administrativo, no gerenciamento das edificações e nos fluxos ligados a idiomas e mídias, sem perder a visão do produto como um todo. Para isso, utilizamos o *board* do GitHub Projects. Os repositórios estão em:

Repositório para o *frontend*

Repositório para o *backend*

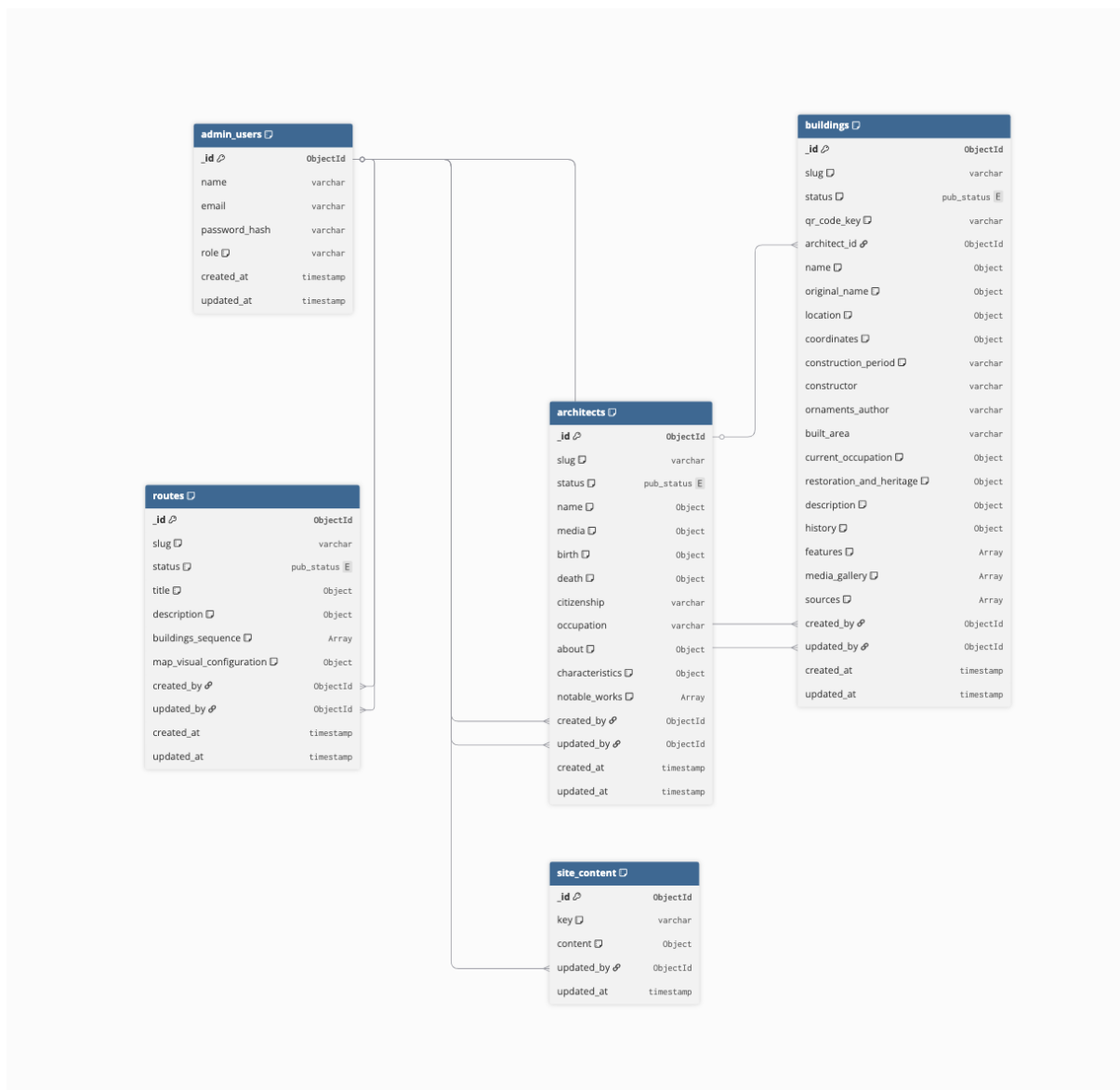
Repositório para a *wiki*

5.2.2 Banco de Dados Utilizado

O banco de dados principal escolhido para o projeto foi o MongoDB Atlas (MongoDB, Inc., 2026). A decisão fez sentido para o nosso contexto porque o produto precisa lidar com conteúdos variados e com possibilidade de evolução frequente, como informações de edificações, materiais multimídia e dados administrativos.

Como o MongoDB Atlas é um serviço gerenciado, ele também ajudou a reduzir esforço operacional com manutenção, backup e disponibilidade. Na prática, isso nos permitiu focar mais na modelagem dos dados e nas regras de negócio do sistema, em vez de gastar energia com configuração de infraestrutura de banco do zero.

Figura 13 – Diagrama do Banco de Dados



Fonte: Diagrama feito pelos AGES II.

5.2.3 Arquitetura Utilizada

A arquitetura do sistema foi definida em três camadas: *frontend*, *backend* e dados/armazenamento. No *frontend*, utilizamos o AWS Amplify (Amazon Web Services, 2026c) para publicar a aplicação web com *deploy* automatizado a partir do repositório. No *backend*, adotamos NestJS (NestJS, 2026) em *container* Docker (Docker, Inc., 2023) rodando no ECS Fargate (Amazon Web Services, 2026a,d), o que simplificou a operação por não exigir gerenciamento direto de servidores.

Na camada de dados e arquivos, utilizamos MongoDB Atlas para persistência principal e Amazon S3 (Amazon Web Services, 2026b) para armazenamento de imagens,

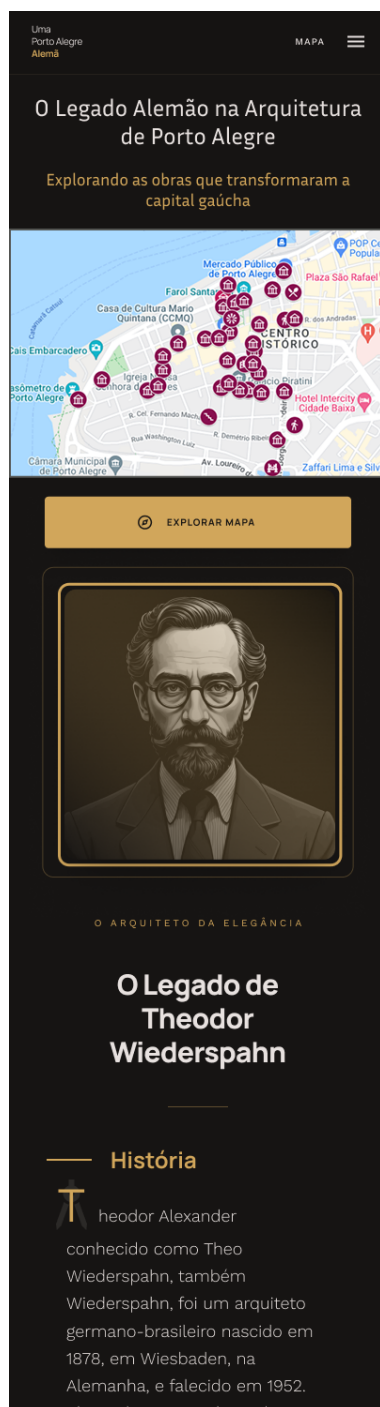
áudios e documentos. Para proteger informações sensíveis, como credenciais e chaves, utilizamos AWS Secrets Manager (Amazon Web Services, 2026e). Esse desenho trouxe um bom equilíbrio entre simplicidade, segurança e escalabilidade, além de facilitar a evolução da plataforma conforme novas demandas do projeto cultural surgirem.

Em termos de fluxo, o usuário acessa o *frontend* no navegador, que envia requisições para a API. O *backend* processa essas requisições e, quando necessário, consulta o MongoDB, manipula arquivos no S3 e busca segredos no Secrets Manager. Depois disso, a resposta retorna ao *frontend* para exibição ao usuário final. A solução final ainda não está implementada, então seguimos com a arquitetura planejada para garantir que o desenvolvimento fique alinhado com as necessidades do projeto e dos *stakeholders*.

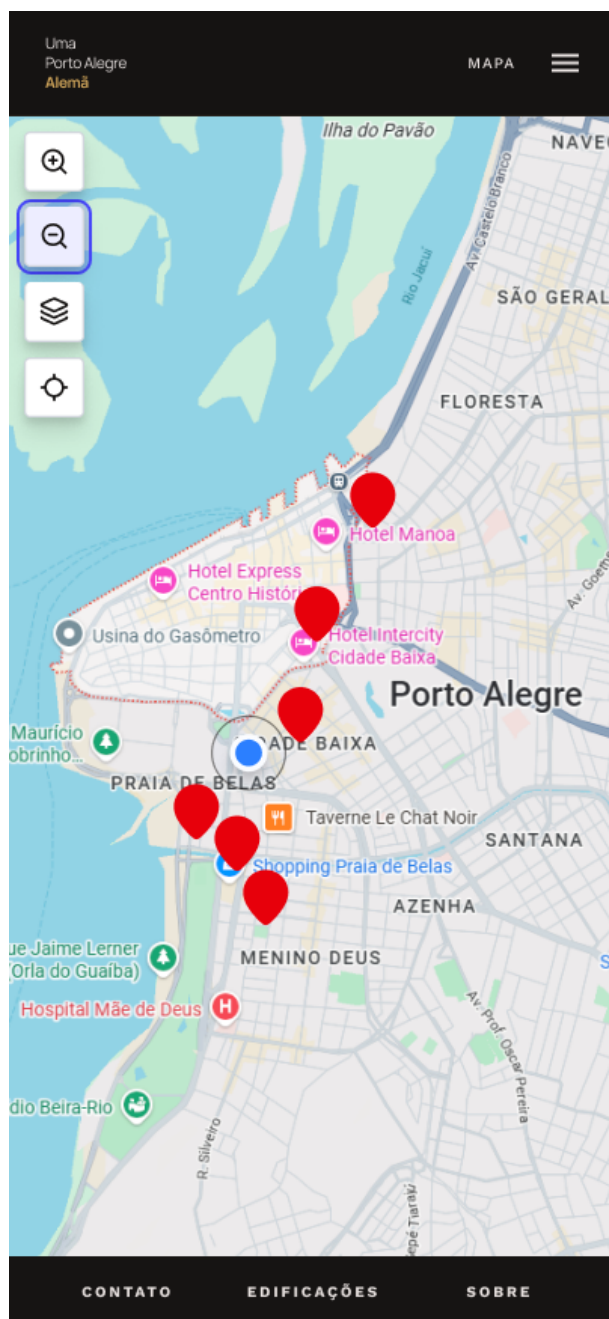
5.2.4 Protótipos das Telas Desenvolvidas

Os protótipos foram desenvolvidos no Figma (Figma, Inc., 2026) e foram parte crucial para alinhar a visão do produto entre os *stakeholders* e o time de desenvolvimento. Eles ajudaram a definir a estrutura das páginas, a navegação e a organização dos conteúdos, além de facilitar o *feedback* e as iterações durante o processo de design. A seguir, apresentamos exemplos de protótipo para ilustrar a proposta visual do *website*:

Figura 14 – Protótipo no Figma - Landing Page

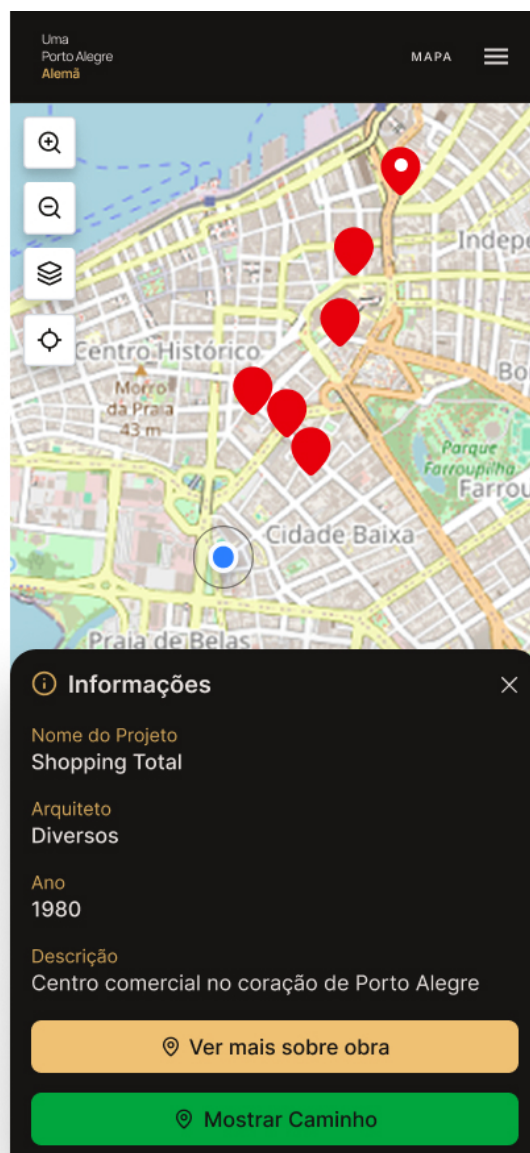


Fonte: Figma do projeto

Figura 15 – Protótipo no Figma - Mapa

Fonte: Figma do projeto

Figura 16 – Protótipo no Figma - Detalhe no Mapa



Fonte: Figma do projeto

Figura 17 – Protótipo no Figma - Detalhe de Edificações



Fonte: Figma do projeto

5.2.5 Tecnologias Utilizadas

A base de desenvolvimento do projeto foi definida com TypeScript (Microsoft, 2023) tanto no *frontend* quanto no *backend*. Essa escolha ajudou a manter maior consistência entre as camadas da aplicação, reduzir erros de tipagem e facilitar a manutenção do código ao longo das sprints.

No *frontend*, utilizamos React (Facebook, 2023a) com Next.js (Vercel, 2026), combinação que nos permitiu estruturar melhor a interface do *website*, organizar rotas de forma mais clara e evoluir as telas com boa reutilização de componentes. No *backend*, utilizaremos NestJS, que trouxe uma estrutura modular alinhada com o padrão que queríamos para o CMS e para as APIs do projeto. No geral, essa *stack* focada em TypeScript, React/Next.js e NestJS deixou o desenvolvimento mais coeso entre *frontend* e *backend* e diminuiu a curva de aprendizado para o time, já que utilizamos as mesmas linguagens e conceitos em ambas as camadas.

5.3 Atividades Desempenhadas Pelo Aluno no Projeto

5.3.1 Sprint 0

Na Sprint 0, meu foco esteve concentrado na estruturação inicial do produto e na organização do trabalho do time. Comecei atuando diretamente na criação de *user stories*, principalmente as relacionadas ao Painel Admin, e fiquei envolvido de forma completa na definição das 7 histórias dessa frente. Esse trabalho foi importante para deixar mais claro o que realmente precisava ser construído no início do projeto e para reduzir ambiguidades na hora de transformar as demandas em tarefas técnicas.

Além das histórias, também criei tarefas voltadas ao gerenciamento de edificações tanto para *frontend* quanto para *backend*. Em paralelo, comecei a mapear tarefas relacionadas a gerenciamento de imagens e idiomas. Essas últimas ainda ficaram em fase de refinamento, porque preferimos amadurecer melhor os critérios antes de levar para execução direta. Mesmo assim, esse levantamento inicial me ajudou a antecipar pontos que poderiam gerar retrabalho mais adiante.

Outra atividade relevante dessa sprint foi a preparação para a reunião com os clientes. Organizei perguntas para conduzir melhor a conversa e, junto com os outros AGES IV, conseguimos obter respostas objetivas sobre o que era prioridade no produto. Esse momento foi decisivo para alinhar expectativas e para validar o direcionamento das histórias do admin, já que boa parte das decisões de escopo dependia desse retorno. A reunião também serviu para fortalecer a conexão com os *stakeholders*, acabei sendo o responsável por conduzir esta conversa, sendo essa a minha primeira experiência com esse tipo de contato direto, o que foi bastante enriquecedor para entender melhor as necessidades do projeto e para desenvolver habilidades de

comunicação e alinhamento e criar uma boa relação com os clientes.

Como dificuldade, no começo eu me senti perdido para definir com clareza quem ficaria responsável por cada parte entre os AGES IV, e isso acabou atrasando um pouco o andamento inicial. A principal lição que levo dessa sprint é a necessidade de distribuir responsabilidades de forma mais assertiva desde o início, com escopos melhor definidos para cada pessoa. Quando a divisão fica clara, o time ganha ritmo mais rápido e as entregas tendem a ficar mais consistentes.

No mais, essa Sprint foi tranquila em termos de execução e tudo parecia ocorrer bem, embora ainda estivéssemos um pouco perdidos, o que é natural para um projeto que está começando. O importante é que aula por aula conseguíamos nos conectar melhor e entender as necessidades do projeto, assim como de cada aluno, o que nos deixou confiantes para o continuamento da jornada.

5.3.2 Sprint 1

Na Sprint 1, dei continuidade ao trabalho de estruturação do produto, mantendo meu foco principal no Painel Admin e no desdobramento técnico das funcionalidades. Segui atuando fortemente na construção e no refinamento das *user stories* do admin, garantindo que os critérios estivessem mais claros para o time de desenvolvimento e alinhados com o que foi discutido com os clientes.

Também mantive minha atuação na criação e ajuste de tarefas para o gerenciamento de edificações em *frontend* e *backend*, além de acompanhar o detalhamento de pontos ligados a imagens e idiomas, que ainda exigiam refinamento. Essa etapa foi importante para transformar discussões mais conceituais em tarefas executáveis, com menor chance de interpretação diferente entre os times.

Nessa Sprint, comecei a perceber uma forte ausência nas nossas *dailies* semanais, o que dificultou muito o andamento do projeto. As comunicações até aconteciam, mas de maneira oculta, os AGES III falavam entre si e não comunicavam decisões nem reportavam algum tipo de avanço, o que me trouxe um certo desconforto. Além disso, não obtivemos muitas respostas em relação aos AGES III sobre o que estava sendo feito, qual era o progresso da tarefa nas quais eles estavam supervisionando, ou até mesmo se existia algum tipo de bloqueio. As cobranças aconteciam e, se houvesse resposta, era sempre algo genérico do tipo "estamos trabalhando nisso",

sem detalhamento do que estava sendo feito, o que me deixou bastante perdido e sem saber como ajudar ou como me organizar para contribuir melhor. Como parte disso, criei também uma planilha no *Excel* para organizar os relatos da *daily* semanal e verificar ausências, para assim ter um histórico e talvez alguma resposta do porquê alguma tarefa não tinha sido entregue.

Apesar disso, alguns AGES III acabaram por terminar algumas das tarefas remanescentes da Sprint, e conseguimos entregar algo viável para o cliente. Tivemos um mapa funcional, com as edificações e as informações básicas, o que foi um avanço importante para o projeto. No entanto, a AGES, por ser um ambiente acadêmico, deveria ser um espaço de aprendizado e desenvolvimento de habilidades, o que não aconteceu, já que alguns alunos da AGES I e II não tiveram interesse em terminar suas tarefas e indagar. No fim, o cliente ficou satisfeito com o que foi entregue, acabei conduzindo a apresentação para os clientes e fiquei feliz com o modo que conduzi, pois em outro projeto passado na Agência, os AGES IV tiveram dificuldades de mostrar o porquê da entrega ter tido débitos técnicos, falando até que era inadmissível. Dessa vez, eu expliquei de maneira clara e objetiva que estamos todos aprendendo e, mesmo com todas as dificuldades dessa sprint, isso serve apenas para nos fortalecer e saber para onde estamos indo.

A principal lição que levo dessa Sprint é a importância de ter uma comunicação mais transparente e frequente entre os AGES III e os AGES IV, para que todos possam acompanhar o andamento do projeto e contribuir de forma mais efetiva. A falta de comunicação clara e de *feedbacks* regulares acabou gerando um ambiente de incerteza, onde não sabíamos exatamente o que estava sendo feito ou quais eram os próximos passos, o que impactou negativamente o ritmo do desenvolvimento. Combinamos de criar canais de ajuda no *Discord* para facilitar a comunicação e o acompanhamento das tarefas, além de estabelecer um compromisso de reportar avanços e bloqueios de forma mais detalhada nas *dailies*, para que todos possam estar alinhados e contribuir melhor para o sucesso do projeto.

5.3.3 Sprint 2

No mínimo uma página contendo tudo que o aluno fez na Sprint 2.

5.3.4 Sprint 3

No mínimo uma página contendo tudo que o aluno fez na Sprint 3.

5.3.5 Sprint 4

No mínimo uma página contendo tudo que o aluno fez na Sprint 4.

5.4 Conclusão

—

6 CONSIDERAÇÕES FINAIS

As considerações finais referem-se a trajetória do aluno no curso, onde se expõe o fechamento da narrativa e são apresentados os resultados alcançados.

Este item é somente para os AGES IV.

Em particular, espera-se neste capítulo:

- contribuições que o curso trouxe para a sua evolução profissional
 - competências (o que) e habilidades desenvolvidas (como), (hardskills e softskills);
 - lições aprendidas (o que deu certo, o que deu errado);
- uma reflexão sobre a visão do aluno sobre a prática da Engenharia de Software, como era no início de sua trajetória, e que visão ele tem hoje;
- eventuais comentários que deseje adicionar;
- sugestão de melhorias, críticas e elogios em relação a AGES.

(No mínimo uma página de relato)

REFERÊNCIAS

AMAZON WEB SERVICES. **Amazon Elastic Container Service (Amazon ECS)**.

[S. l.: s. n.], 2026. Acessado em: 19/04/2026. Disponível em:

<https://aws.amazon.com/ecs/>.

AMAZON WEB SERVICES. **Amazon Simple Storage Service (Amazon S3)**.

[S. l.: s. n.], 2026. Acessado em: 19/04/2026. Disponível em:

<https://aws.amazon.com/s3/>.

AMAZON WEB SERVICES. **AWS Amplify**. [S. l.: s. n.], 2026. Acessado em:

19/04/2026. Disponível em: <https://aws.amazon.com/amplify/>.

AMAZON WEB SERVICES. **AWS Fargate**. [S. l.: s. n.], 2026. Acessado em:

19/04/2026. Disponível em: <https://aws.amazon.com/fargate/>.

AMAZON WEB SERVICES. **AWS Secrets Manager**. [S. l.: s. n.], 2026. Acessado

em: 19/04/2026. Disponível em: <https://aws.amazon.com/secrets-manager/>.

AMAZON.COM, INC. **Amazon Web Services (AWS): Cloud Computing Services**.

[S. l.: s. n.], 2025. Acessado em: 16/11/2025. Disponível em:

<https://aws.amazon.com/>.

DOCKER, INC. **Docker: Empowering App Development for Developers**.

[S. l.: s. n.], 2023. Acessado em: 05/07/2024. Disponível em:

<https://www.docker.com/>.

FACEBOOK. **React**. [S. l.: s. n.], 2023. Acessado em: 05/07/2024. Disponível em:

<https://reactjs.org/>.

FACEBOOK. **React Native**. [S. l.: s. n.], 2023. Acessado em: 05/07/2024. Disponível

em: <https://reactnative.dev/>.

FIGMA, INC. **Figma: The Collaborative Interface Design Tool**. [S. l.: s. n.], 2026.

Acessado em: 19/04/2026. Disponível em: <https://www.figma.com/>.

GIT COMMUNITY. **Git: Distributed Version Control System**. [S. l.: s. n.], 2025.

Acessado em: 16/11/2024. Disponível em: <https://git-scm.com/>.

GOSLING, James. **The Java Language Specification**. 3rd. [S. l.]: Addison-Wesley Professional, 2018. Acessado em: 05/07/2024. ISBN 0133260224. Disponível em:

<https://docs.oracle.com/javase/specs/jls/se8/html/>.

MATERIAL-UI TEAM. **Material-UI**. [S. l.: s. n.], 2023. Acessado em: 05/07/2024. Disponível em: <https://mui.com/>.

MICROSOFT. **TypeScript: JavaScript with syntax for types**. [S. l.: s. n.], 2023. Acessado em: 05/07/2024. Disponível em: <https://www.typescriptlang.org/>.

MONGODB, INC. **MongoDB Atlas**. [S. l.: s. n.], 2026. Acessado em: 19/04/2026. Disponível em: <https://www.mongodb.com/atlas>.

MONGODB, INC. **MongoDB: The Developer Data Platform**. [S. l.: s. n.], 2024. Acessado em: 14/09/2025. Disponível em: <https://www.mongodb.com/>.

NESTJS. **NestJS: A Progressive Node.js Framework**. [S. l.: s. n.], 2026. Acessado em: 19/04/2026. Disponível em: <https://nestjs.com/>.

PIVOTAL SOFTWARE. **Spring Boot**. [S. l.: s. n.], 2023. Acessado em: 05/07/2024. Disponível em: <https://spring.io/projects/spring-boot>.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL: The World's Most Advanced Open Source Relational Database**. [S. l.: s. n.], 2024. Acessado em: 14/09/2025. Disponível em: <https://www.postgresql.org/>.

POSTMAN. **Postman**. [S. l.: s. n.], 2023. Acessado em: 05/07/2024. Disponível em: <https://www.postman.com/>.

VERCEL. **Next.js**. [S. l.: s. n.], 2026. Acessado em: 19/04/2026. Disponível em: <https://nextjs.org/>.

WORLD WIDE WEB CONSORTIUM (W3C). **CSS: Cascading Style Sheets**. [S. l.: s. n.], 2023. Acessado em: 05/07/2024. Disponível em: <https://www.w3.org/Style/CSS/Overview.en.html>.

WORLD WIDE WEB CONSORTIUM (W3C). **HTML: HyperText Markup Language**. [S. l.: s. n.], 2023. Acessado em: 05/07/2024. Disponível em: <https://html.spec.whatwg.org/multipage/>.

APÊNDICES

APÊNDICE A – Exemplo1: Análise dos relatórios mensais de uso do serviço de renovação de empréstimos.